



第5章 中央处理器I



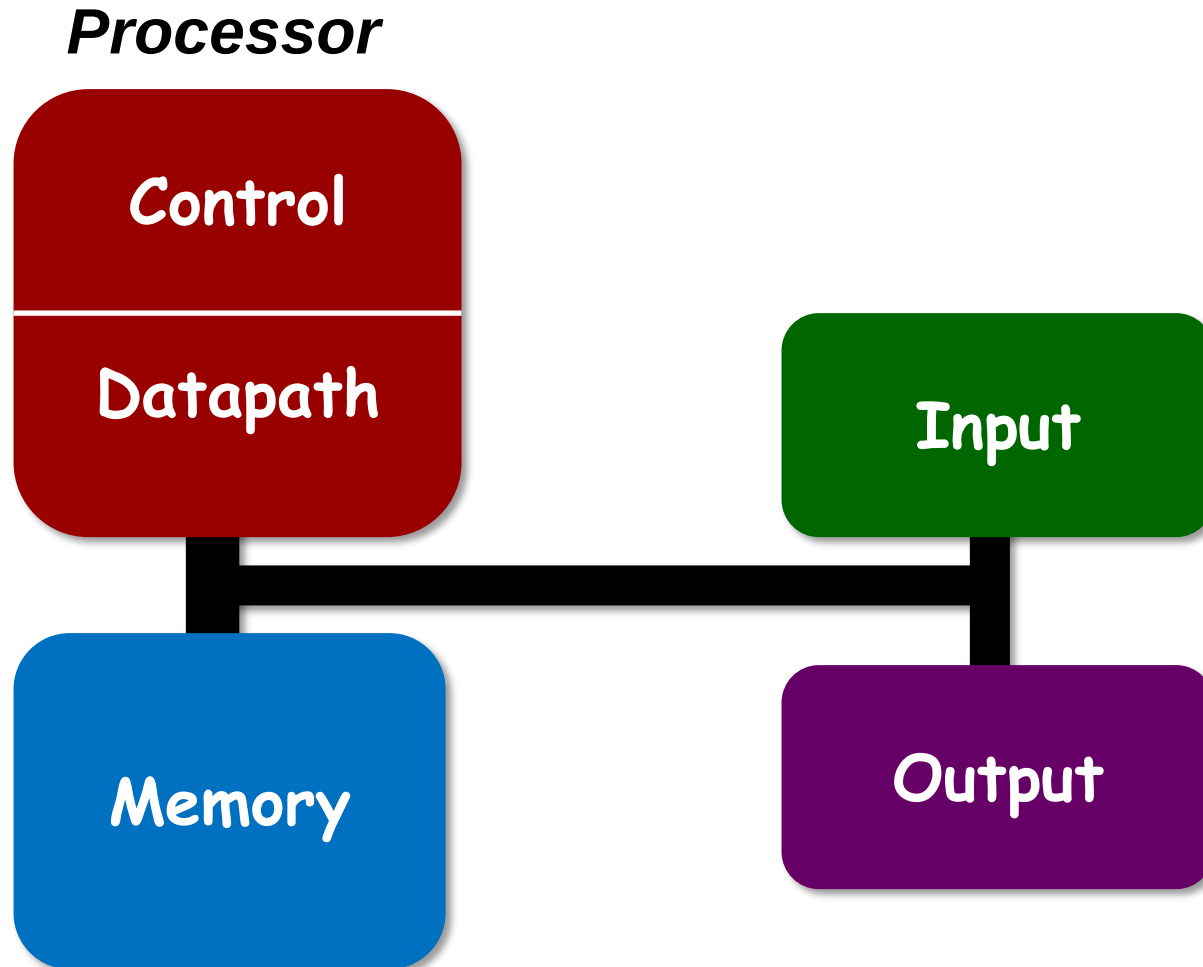


5.1 CPU的功能和组成





计算机的五个基本部件





处理器的功能

■ 指令控制

控制程序严格按照规定的顺序执行

■ 操作控制

处理器根据从内存取出的每条指令产生相应的操作信号，送往相应的部件，控制这些部件按指令的要求进行动作

■ 时间控制

对各种操作实施时间上的控制

■ 数据加工

对数据进行算术运算和逻辑运算处理

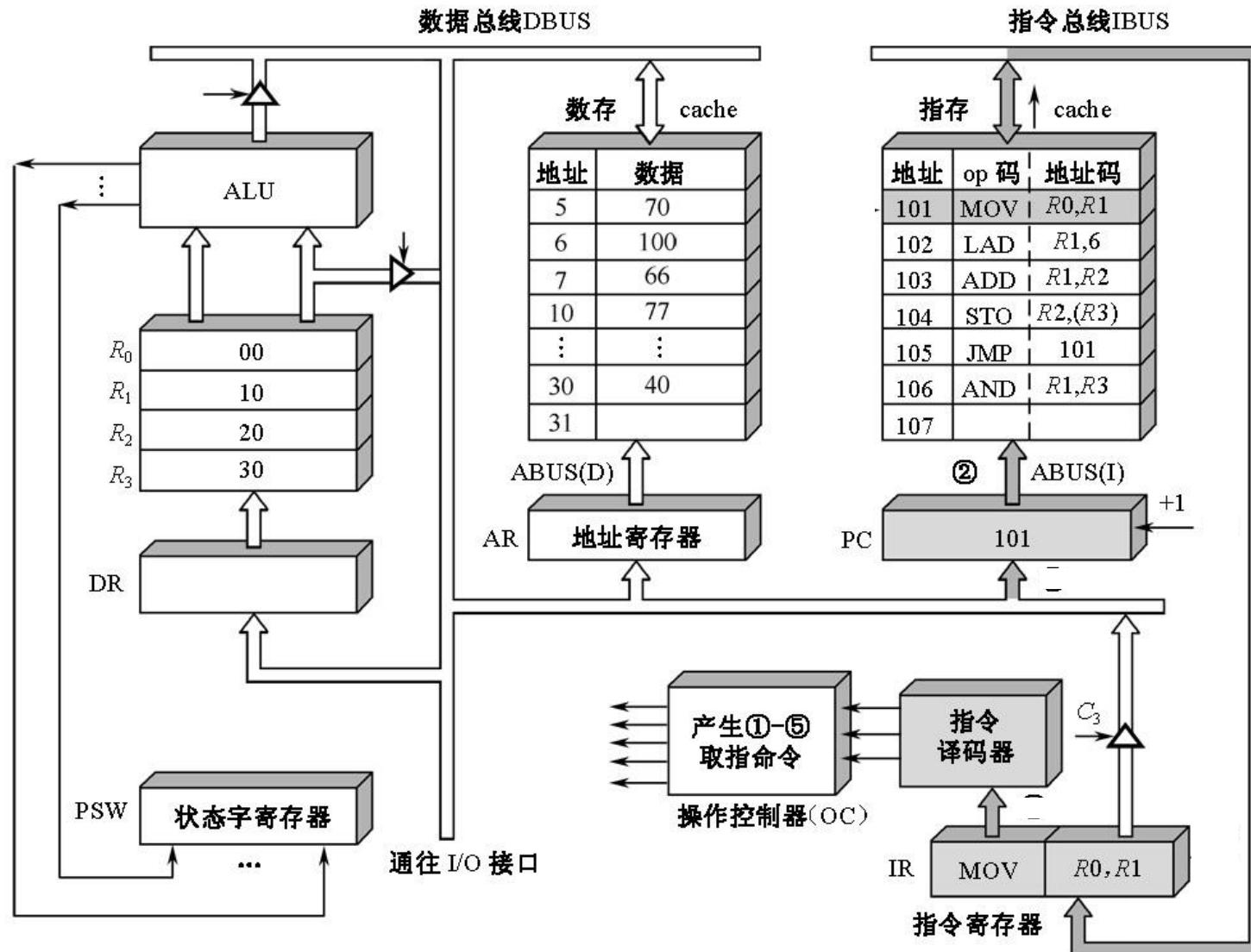
■ 中断处理

对计算机中出现的异常情况和特殊请求进行处理





CPU模型

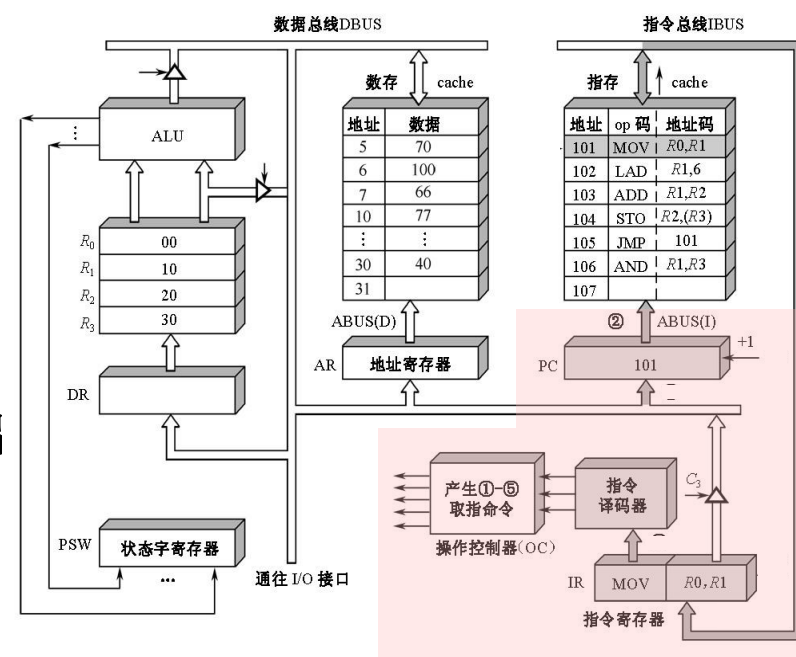




CPU的基本组成 (1)

■ 控制器

- ◆ 由程序计数器、指令寄存器、指令译码器、操作控制器和时序产生器组成
- ◆ 完成指挥和协调整个计算机系统的操作
- ◆ 主要功能
 - 从指令cache中取出一条指令，并指出下一条指令在指令cache中的位置
 - 对指令进行译码或测试，产生相应的操作控制信号，以便启动规定的动作
 - 指挥并控制CPU、内存和输入/输出设备之间数据流动的方向

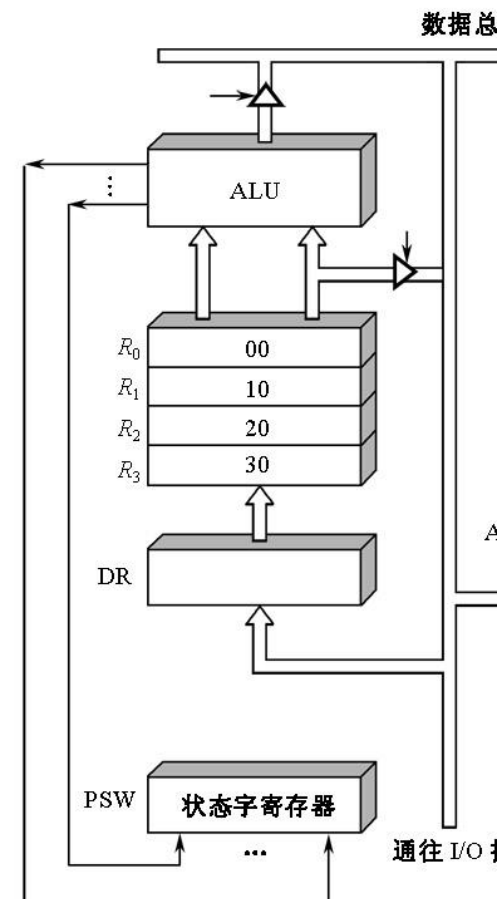




CPU的基本组成 (2)

■ 运算器

- ◆ 由算术逻辑单元(ALU)、累加寄存器、数据缓冲寄存器DR和程序状态字寄存器PSW组成
- ◆ 是数据加工处理部件。运算器接受控制器的命令而进行动作，它是执行部件
- ◆ 主要功能
 - 执行所有的算术运算
 - 执行所有的逻辑运算，并进行逻辑测试，如零值测试或两个值的比较





CPU中的主要寄存器 (1)

- 数据缓冲寄存器 (DR)
- 指令寄存器 (IR)
- 程序计数器 (PC)
- 地址寄存器 (AR)
- 通用寄存器 (R0~R3)
- 程序状态字寄存器 (PSWR)





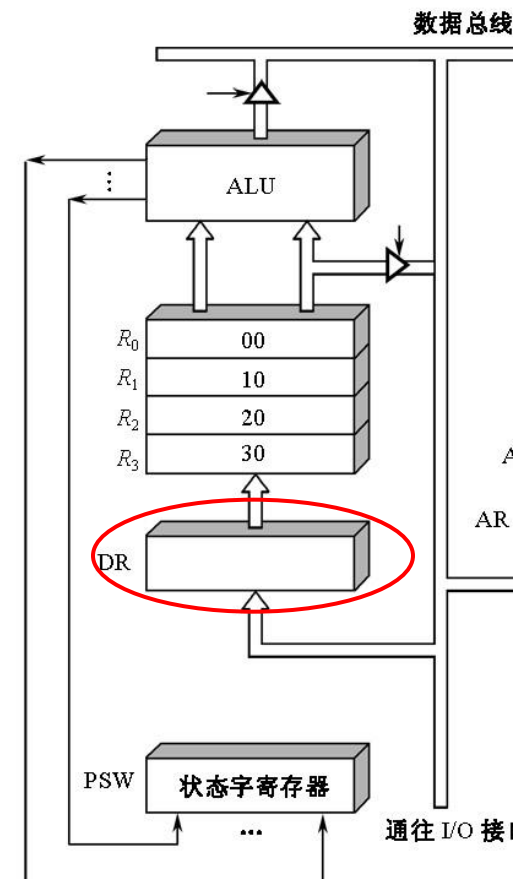
CPU中的主要寄存器 (2)

■ 数据缓冲寄存器 (DR)

◆ 用来暂存从数据cache读出的一个数据字，或来自外部接口的一个数据字，或ALU的运算结果

◆ 作用

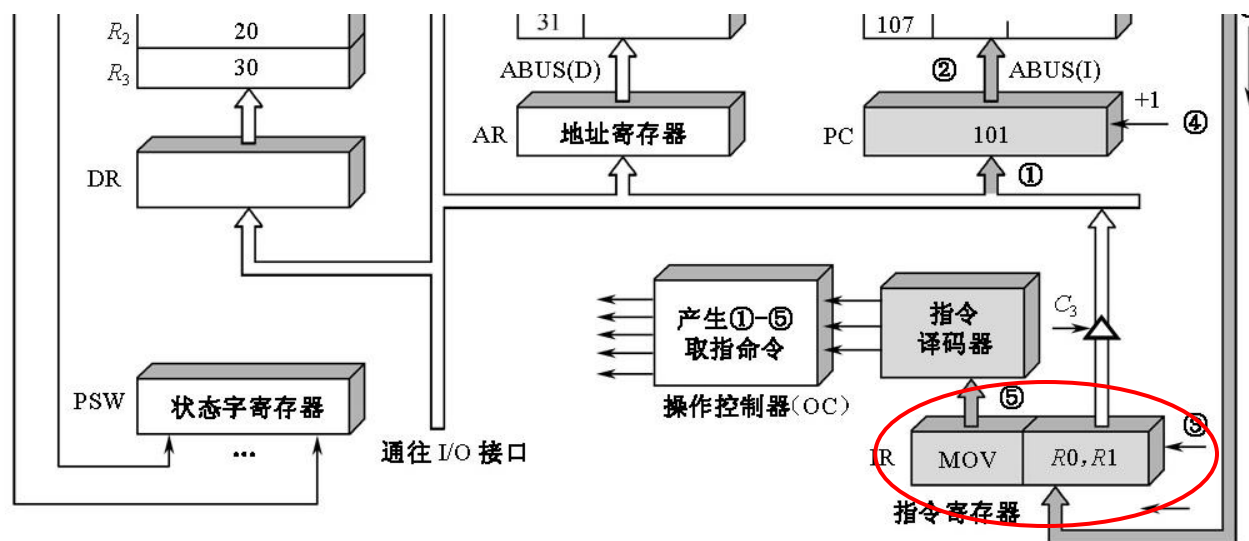
- 作为ALU运算结果和通用寄存器之间信息传送中时间上的缓冲
- 补偿CPU和内存、外设之间在操作速度上的差别



CPU中的主要寄存器 (3)

■ 指令寄存器 (IR)

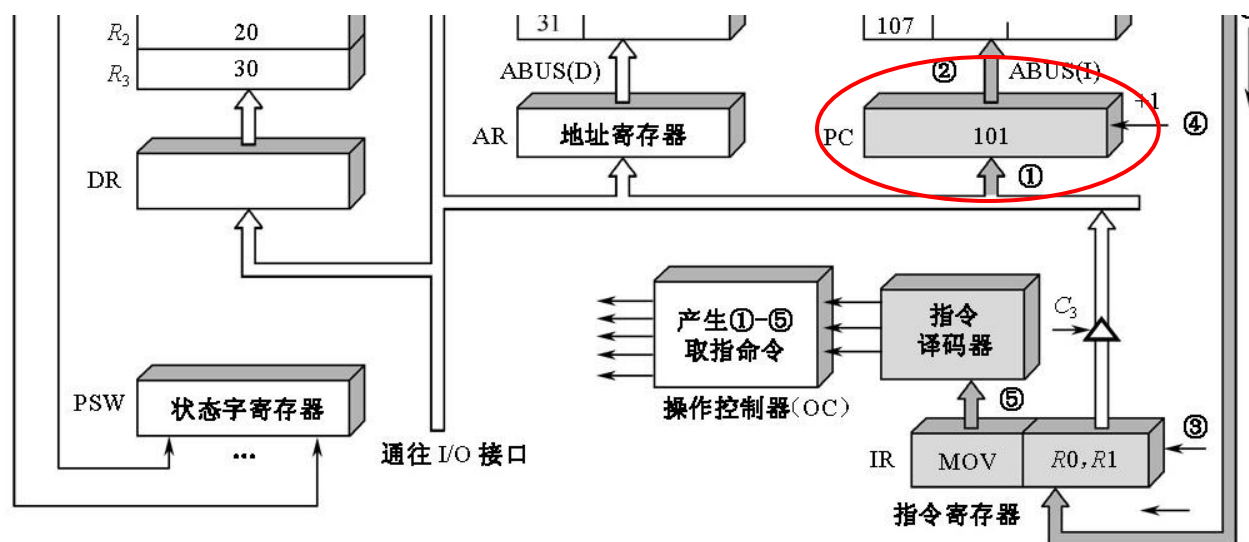
- ◆ 用来保存当前正在执行的一条指令
- ◆ 指令寄存器中操作码字段的输出就是指令译码器的输入
- ◆ 操作码经译码器译码后，即可向操作控制器发出特定的操作信号



CPU中的主要寄存器 (4)

■ 程序计数器 (PC)

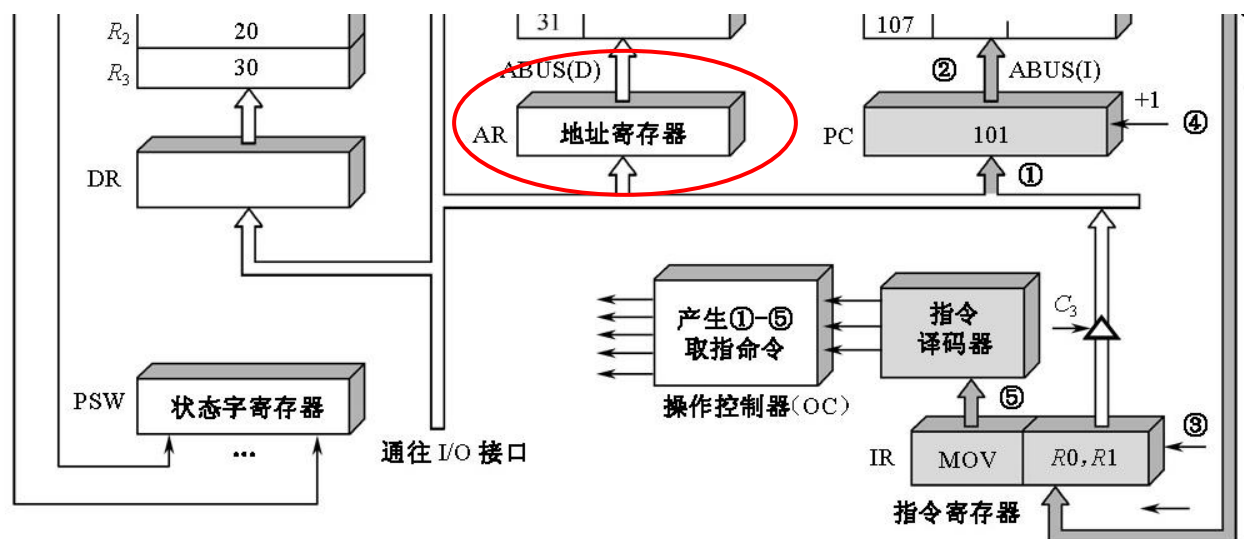
- ◆ 也称为指令计数器，用来确定下一条指令的地址
- ◆ 在程序开始执行前，将程序的第一条指令所在的内存单元地址送入PC
- ◆ 当执行指令时，CPU将自动修改PC的内容。PC的值始终是将要执行的下一条指令地址



CPU中的主要寄存器 (5)

■ 地址寄存器 (AR)

- ◆ 存放处理器当前所访问数据cache中单元的地址
- ◆ 地址寄存器的结构与数据缓冲寄存器、指令寄存器一样，使用单纯的寄存器结构





CPU中的主要寄存器 (6)

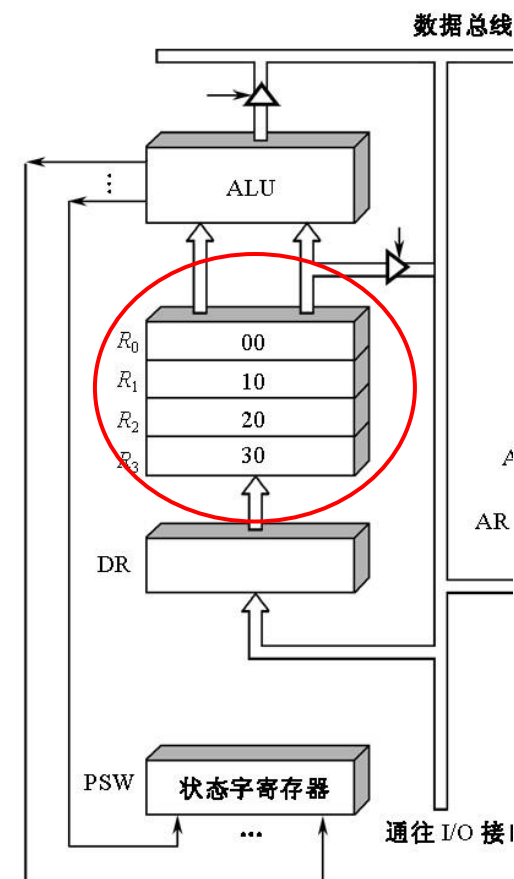
■ 通用寄存器 (R0~R3)

◆ 通用寄存器有4个，R0、R1、R2和R3

◆ 功能

- 当运算器的算术逻辑单元(ALU)执行算术或逻辑运算时，为ALU提供一个暂存数据的工作区
- 累加寄存器暂时存放ALU运算的结果信息

◆ 处理器不同，通用寄存器数目也不相同。数目可从十几个到几十个。





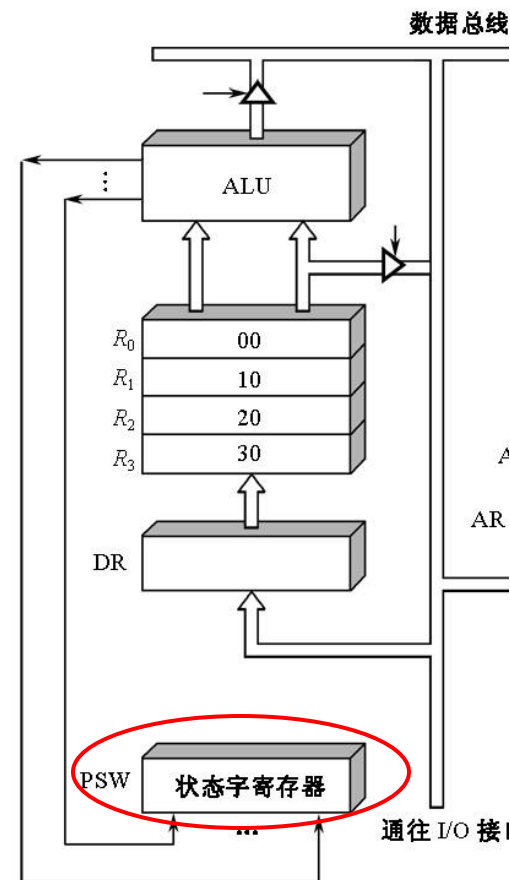
CPU中的主要寄存器 (7)

■ 程序状态字寄存器 (PSWR)

- ◆ 保存根据算术指令、逻辑指令等运行的结果而设定的各种条件码内容
 - 如运算结果进位标志(C), 运算结果溢出标志(V), 运算结果为零标志(Z), 运算结果为负标志(N)等
 - 每个标志位通常由1位触发器保存
- ◆ 保存中断和系统工作状态等信息
 - 处理器能及时了解机器运行状态和程序运行状态

■ 数据通路

- ◆ 各寄存器之间、寄存器与ALU之间传送数据的通路称为数据通路
- ◆ 信息从什么地方开始, 中间经过哪个寄存器或多路开关, 最后传送到哪个寄存器, 都要加以控制
- ◆ 在各寄存器之间建立数据通路的任务, 是由称为操作控制器的部件来完成的



操作控制器与时序产生器

■ 操作控制器

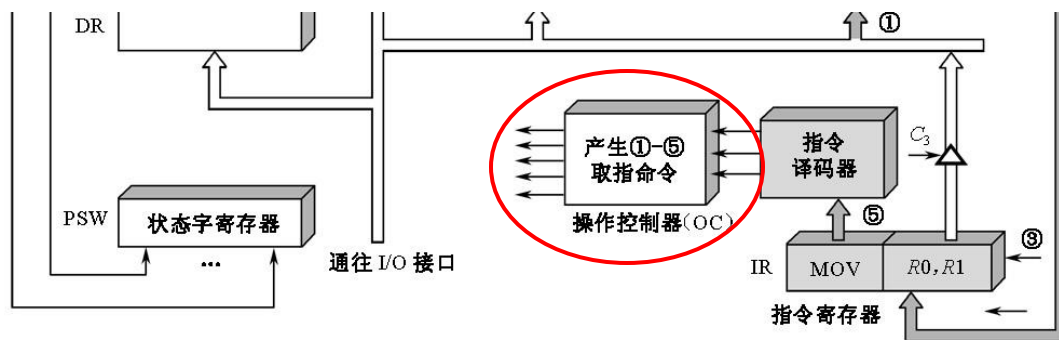
◆ 功能

根据指令操作码和时序信号，产生各种操作控制信号，以便正确地建立数据通路，从而完成取指令和执行指令的操作

◆ 根据设计方法不同，操作控制器可分为：

- 时序逻辑型：采用时序逻辑技术来实现。称为**硬布线控制器**
- 存储逻辑型：采用存储逻辑来实现。称为**微程序控制器**

■ 时序产生器：对各种操作信号进行时间上先后顺序的控制





5.2 指令周期



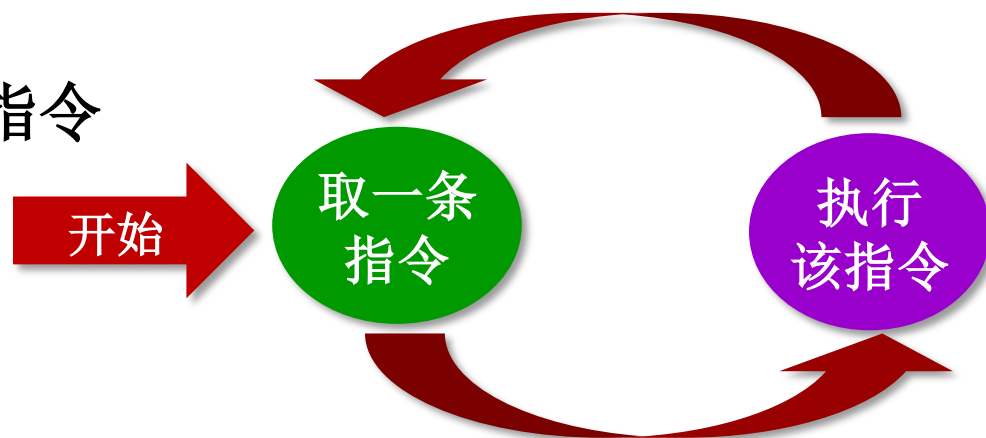
基本概念（1）

■ 冯·诺依曼结构的计算机

即存储程序式计算机，即在程序运行之前，将程序和数据存放到内存中。

■ 执行程序：

- ◆ 将程序首地址送入到程序计数器PC中
- ◆ 从内存（**cache**）中取出一条指令，执行指令
- ◆ 形成下一条待执行指令的地址
- ◆ 自动地连续执行指令
- ◆ 直到程序的最后一条指令





基本概念 (2)

■ 指令周期

- ◆ 指令周期是取指令、分析指令并执行指令所需的总时间
- ◆ 由于各种指令的操作功能不同，因此各种指令的指令周期长度是不相同的
- ◆ 一个指令周期通常包含若干个CPU周期

■ CPU周期（也称机器周期）

- ◆ 是指内存中读取一个指令字的最短时间
- ◆ 一个CPU周期通常包含若干个时钟周期

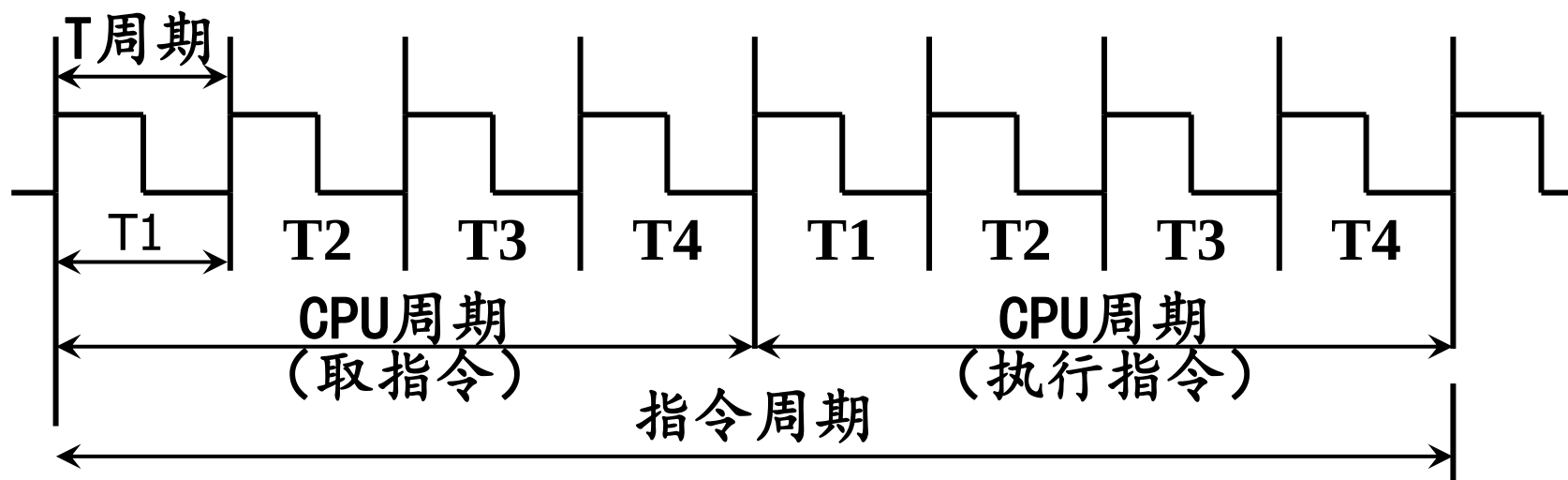
■ 时钟周期

- ◆ 是处理操作的最基本单位，也称为节拍脉冲或T周期





时钟周期、CPU周期、指令周期





程序示例

■ 六条典型指令组成的一个程序

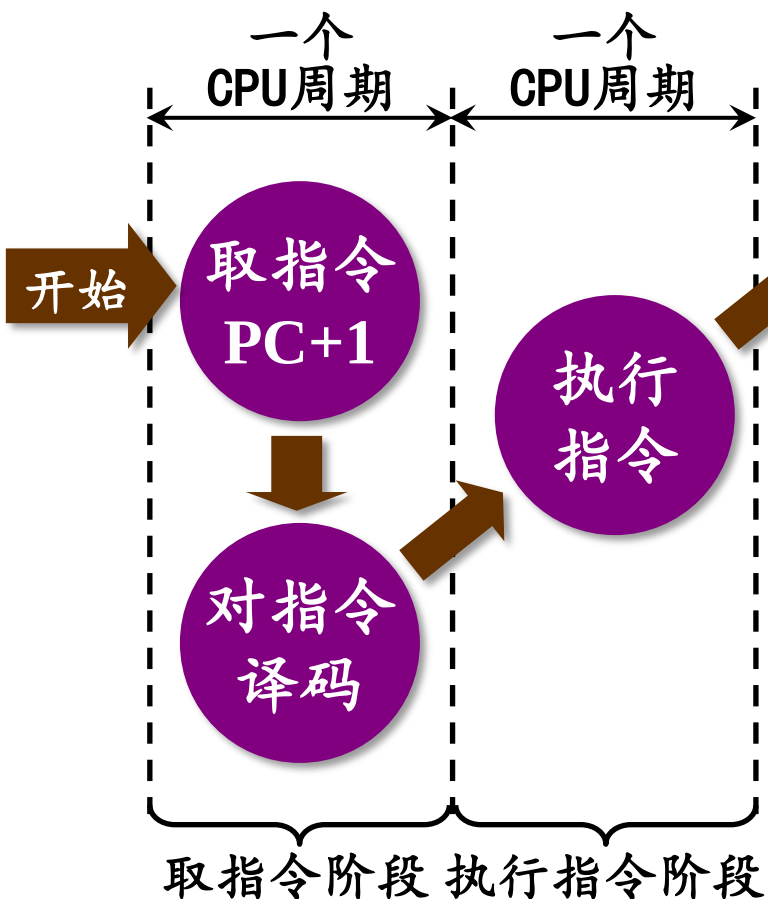
执行前 (R0)=0, (R1)=10, (R2)=20, (R3)=30

八进制地址	助 记 符	说明
101	MOV R1, R0	(R1)→R0
102	LAD R1, 6	(6)→R1
103	ADD R1, R2	(R1)+(R2) →R2
104	STO R2, (R3)	(R2) →(R3)
105	JMP 101	转跳至101
106	AND R1, R3	(R1)^(R3) →R3
5	70	数据
6	100	
7	66	
10	77	
30	40 (120)	





MOV指令的指令周期

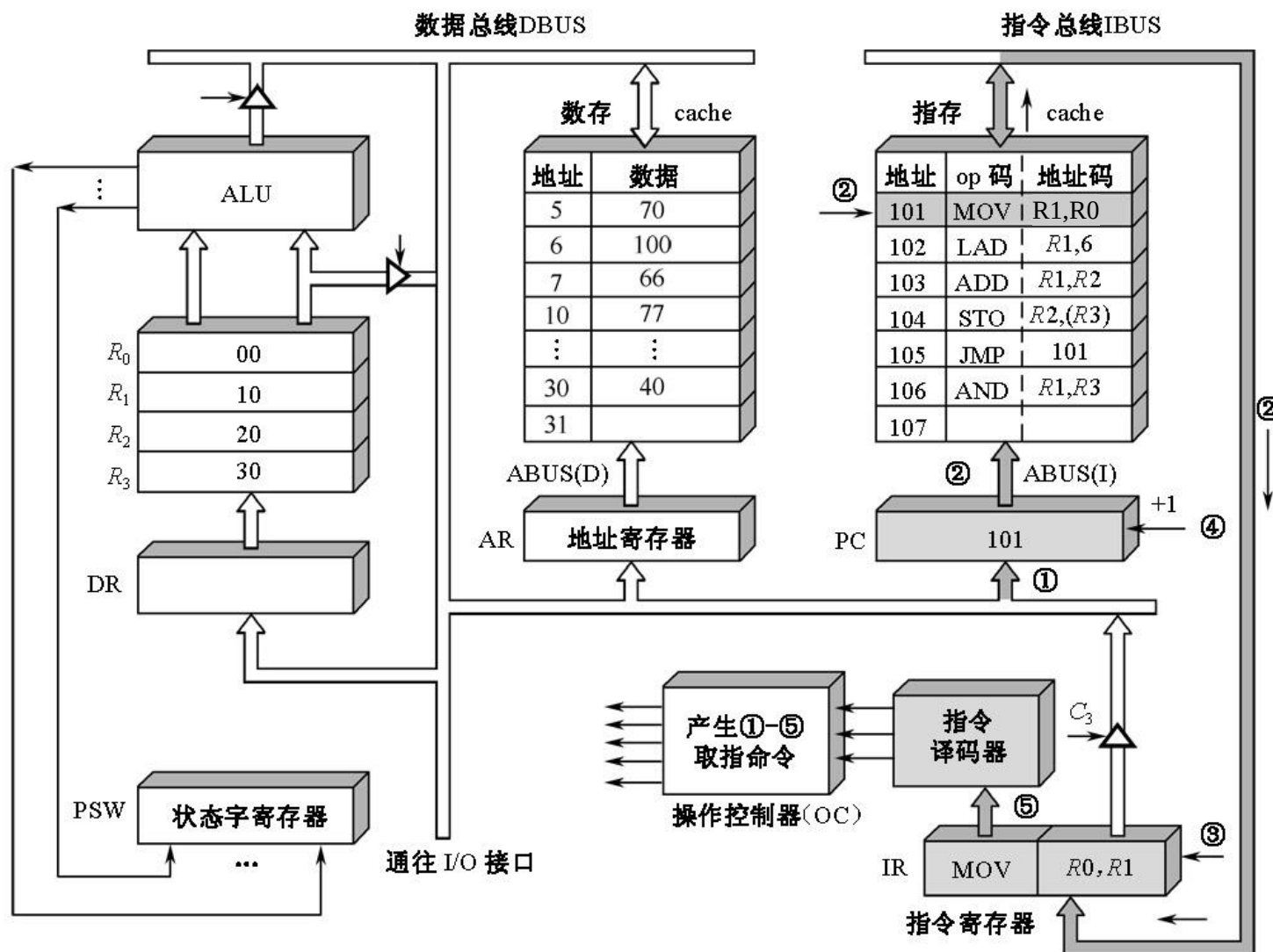


- MOV是一条RR型指令，需要两个CPU周期
- 取指阶段
 - ◆ 从内存取出指令
 - ◆ PC+1，为取下条指令做好准备
 - ◆ 指令译码或测试
- 执行指令阶段
 - ◆ 完成指令所要求的操作
 - ◆ 即完成通用寄存器R0、R1之间的数据传送操作





MOV指令取指周期

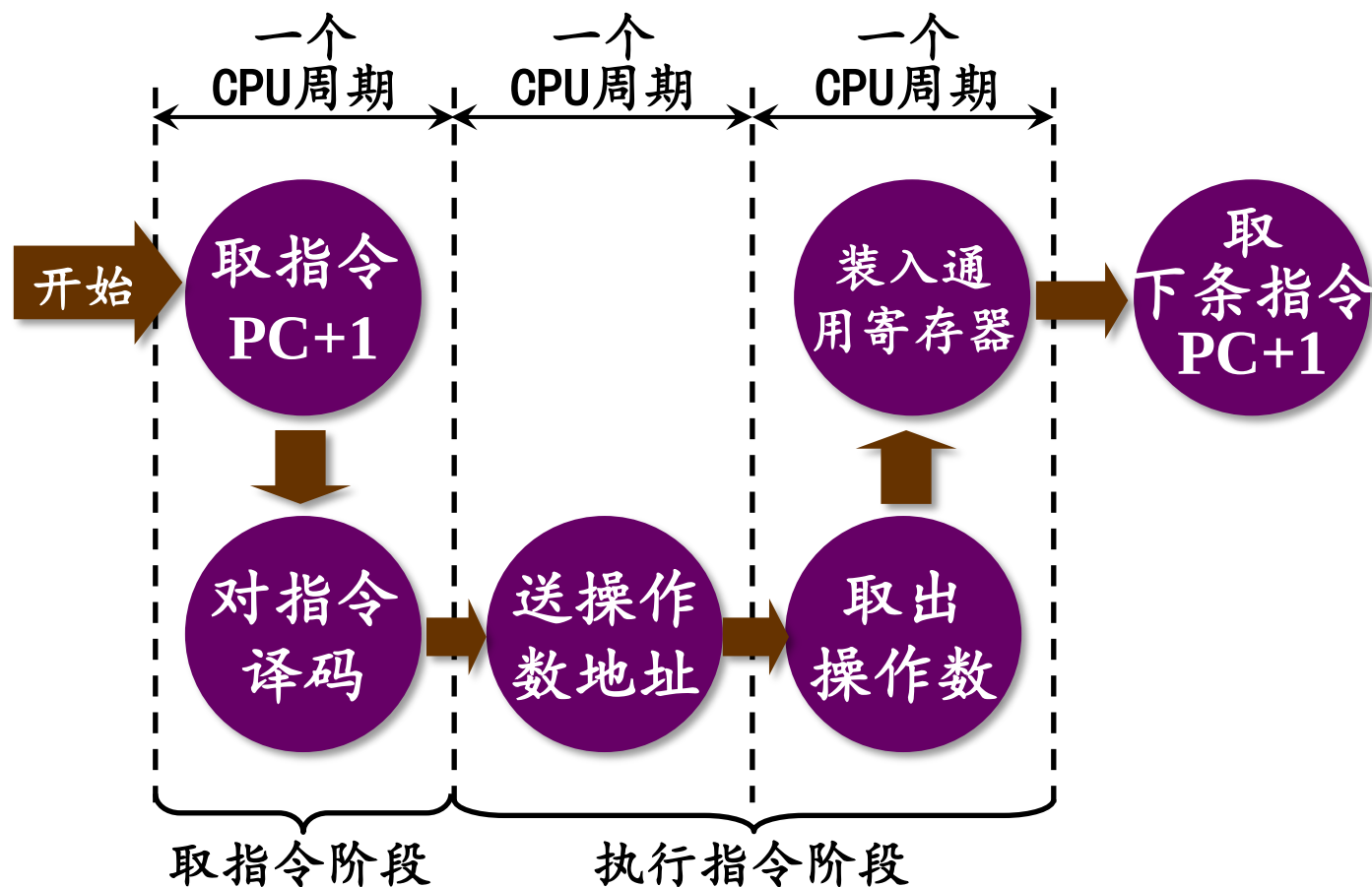






LAD指令的指令周期 (1)

- LAD指令是一条RS型指令
- 由三个CPU周期组成





LAD指令的指令周期（2）

■ 取指阶段

- ◆ 从内存取出指令
- ◆ $PC+1$ ，为取下条指令做好准备
- ◆ 指令译码

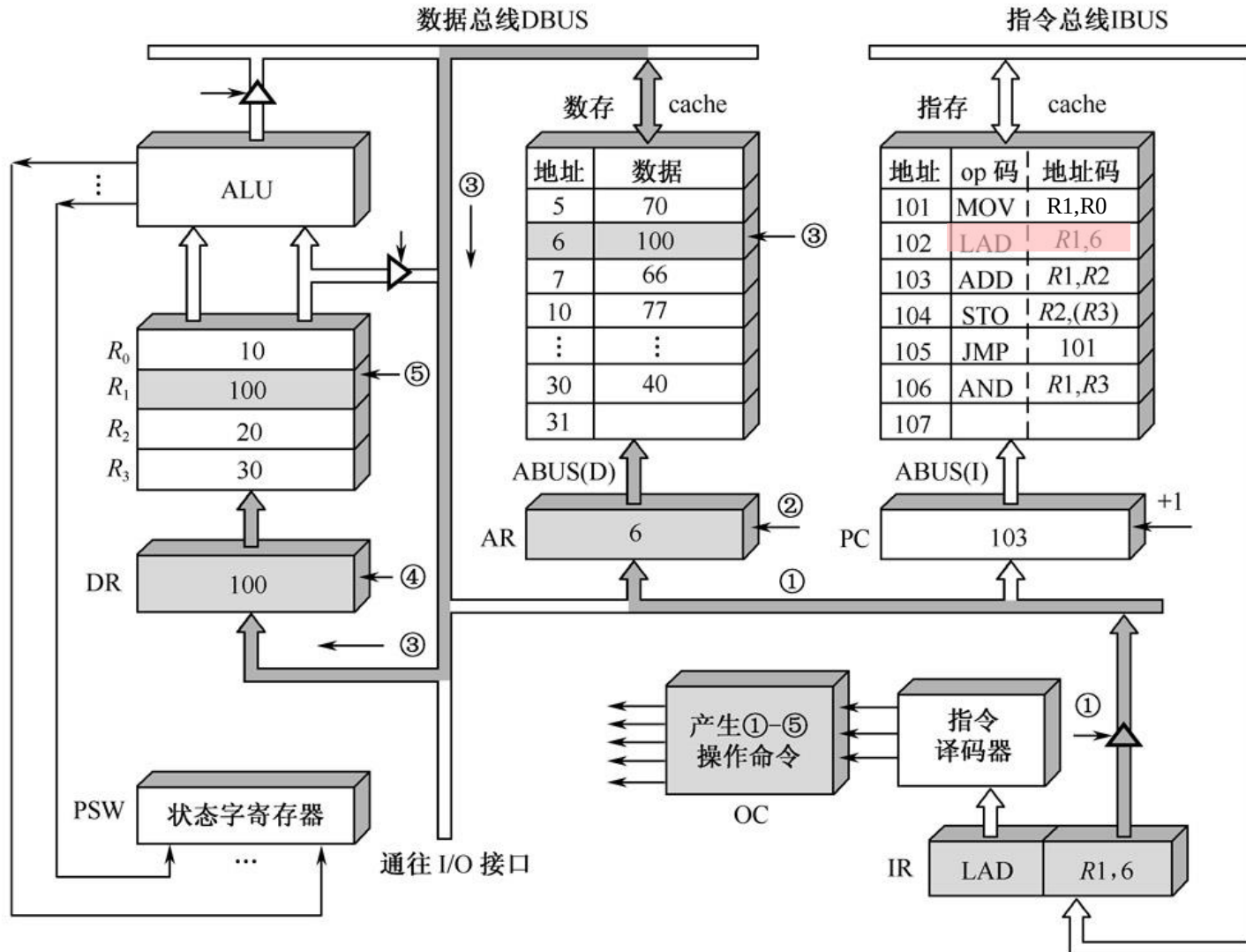
■ 执行指令阶段

- ◆ 将指令中的直接地址6 装入地址寄存器AR中
- ◆ 发出读命令，将数据存储器读出的数据装入DR中
- ◆ 将DR中的数据装入通用寄存器R1中



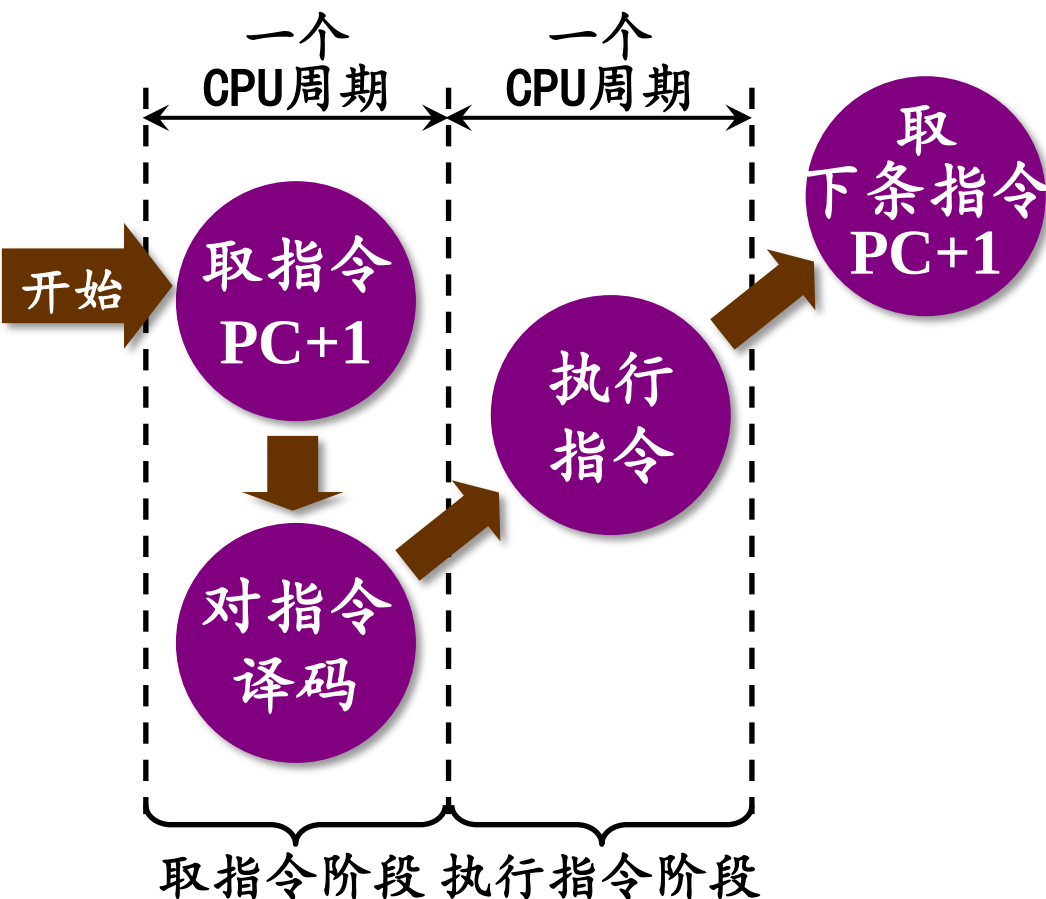


LAD指令执行周期



ADD指令的指令周期

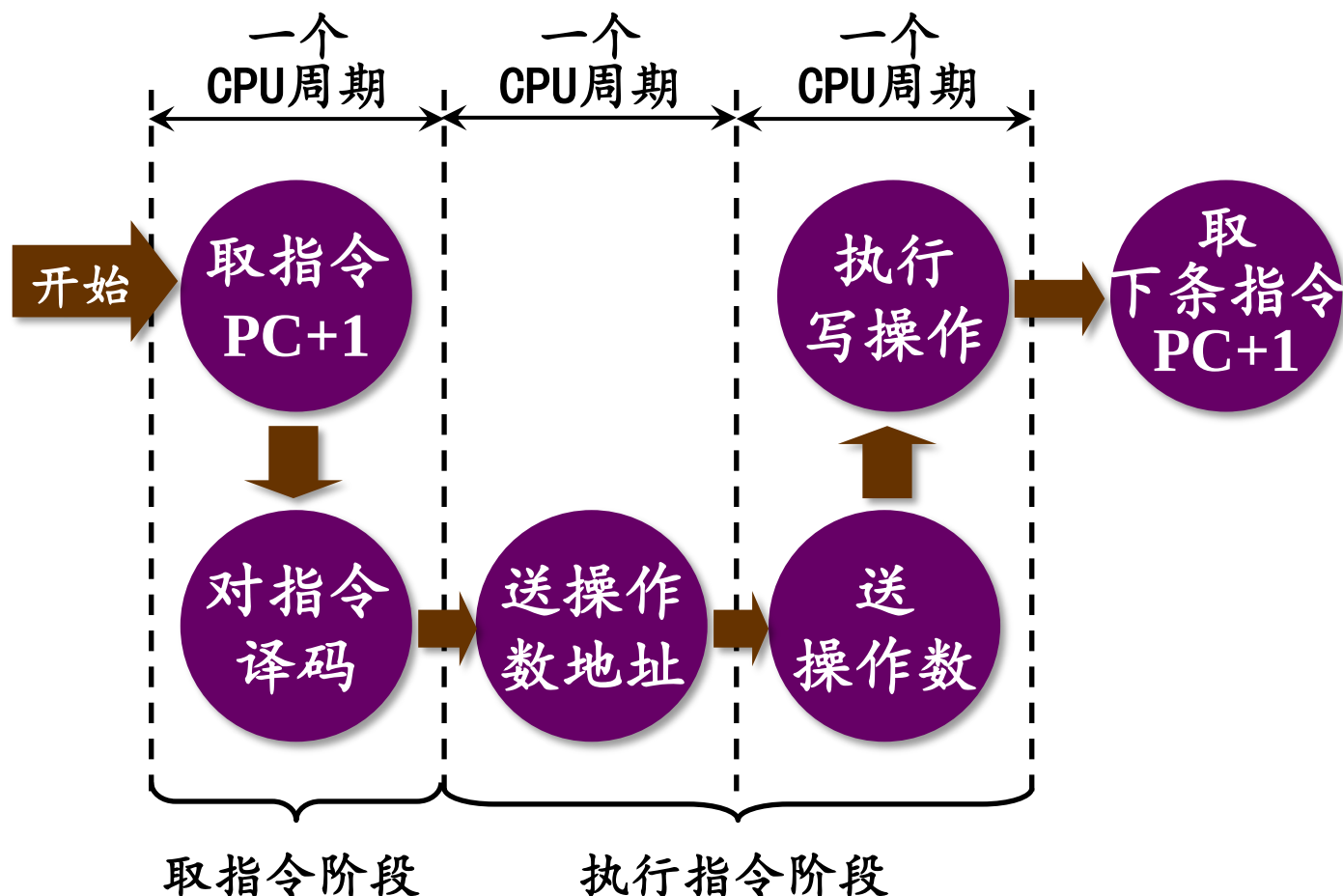
- ADD是一条RR型指令。需要两个CPU周期
- 取指阶段
 - ◆ 从内存取出指令
 - ◆ PC+1, 为取下条指令做好准备
 - ◆ 指令译码
- 执行指令阶段
 - ◆ 完成指令所要求的操作
 - ◆ 即完成通用寄存器R1、R2内容相加, 其结果送R2





STO指令的指令周期 (1)

- STO指令是RS型指令，是访内存的存数指令，由三个CPU周期组成。





STO指令的指令周期（2）

■ 取指阶段

- ◆ 从内存取出指令
- ◆ $PC+1$ ，为取下条指令做好准备
- ◆ 指令译码

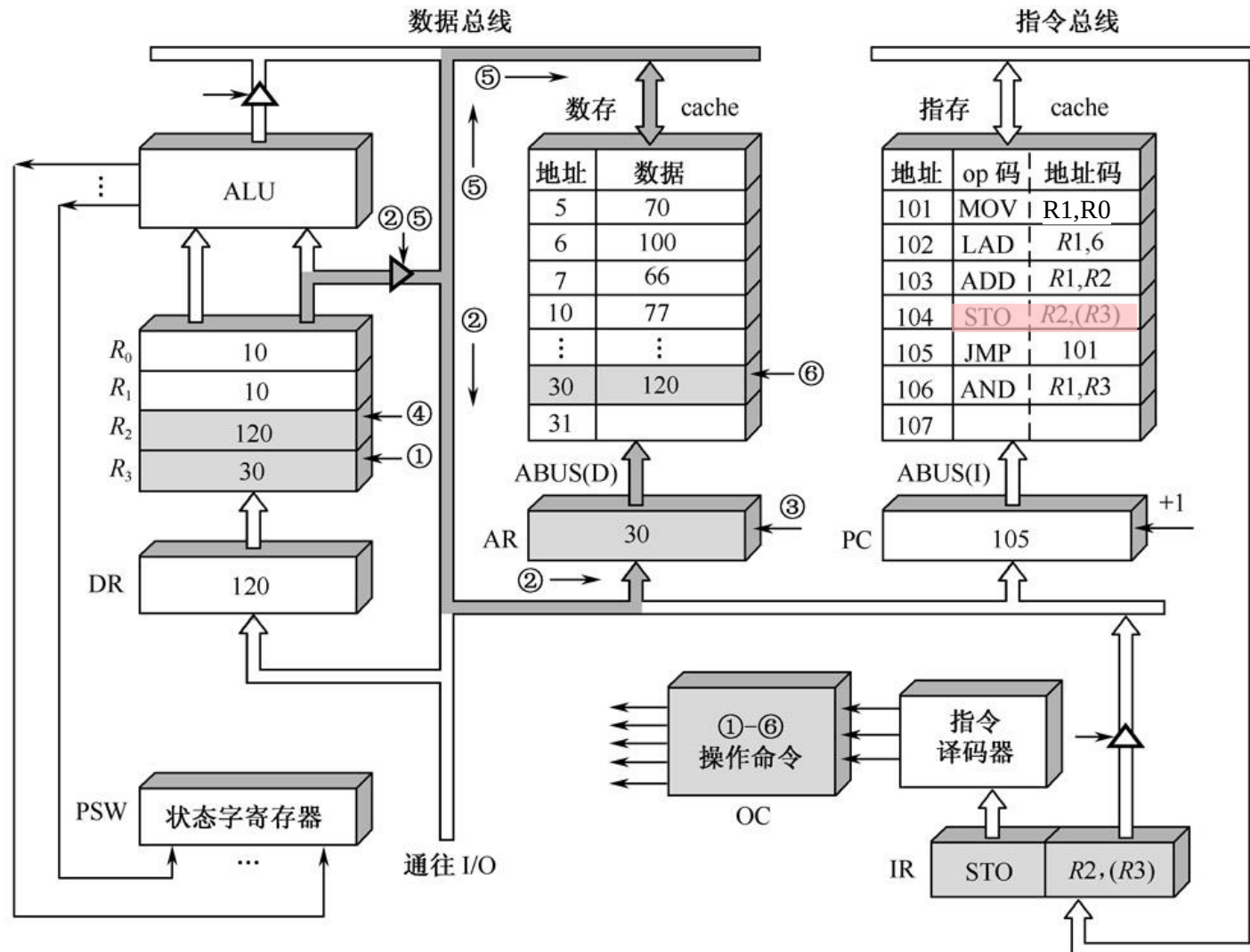
■ 执行指令阶段

- ◆ 将通用寄存器R3的内容读出，装入地址寄存器AR中
- ◆ 将通用寄存器R2的内容读出，将数据放到数据总线上
- ◆ 发出写命令，将数据120写入到30号单元中





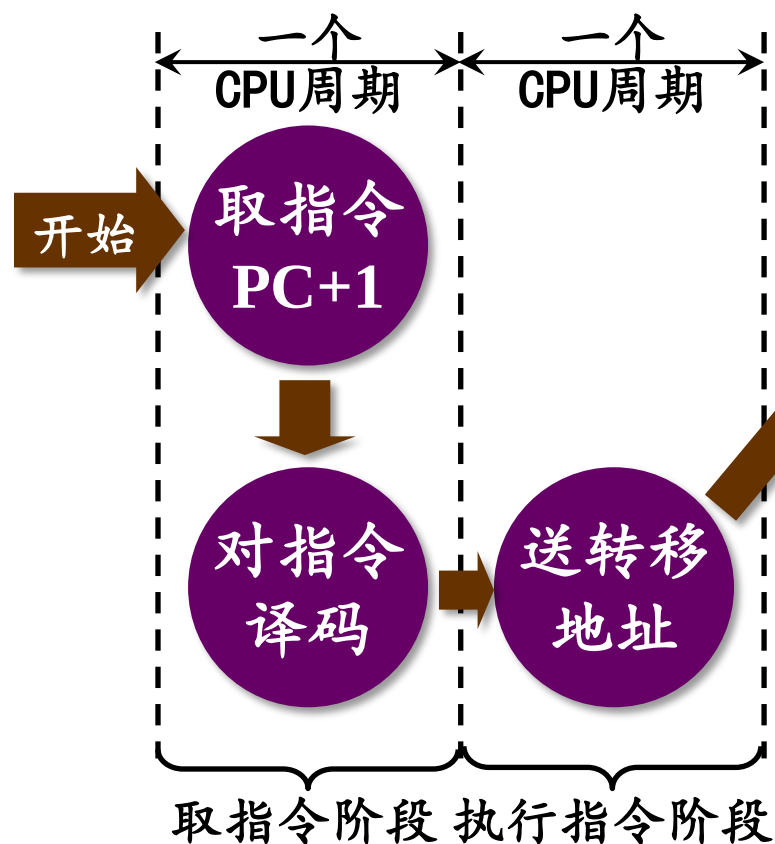
STO指令执行周期





JMP指令的指令周期

- JMP指令是一条无条件转移指令
- 由两个CPU周期组成

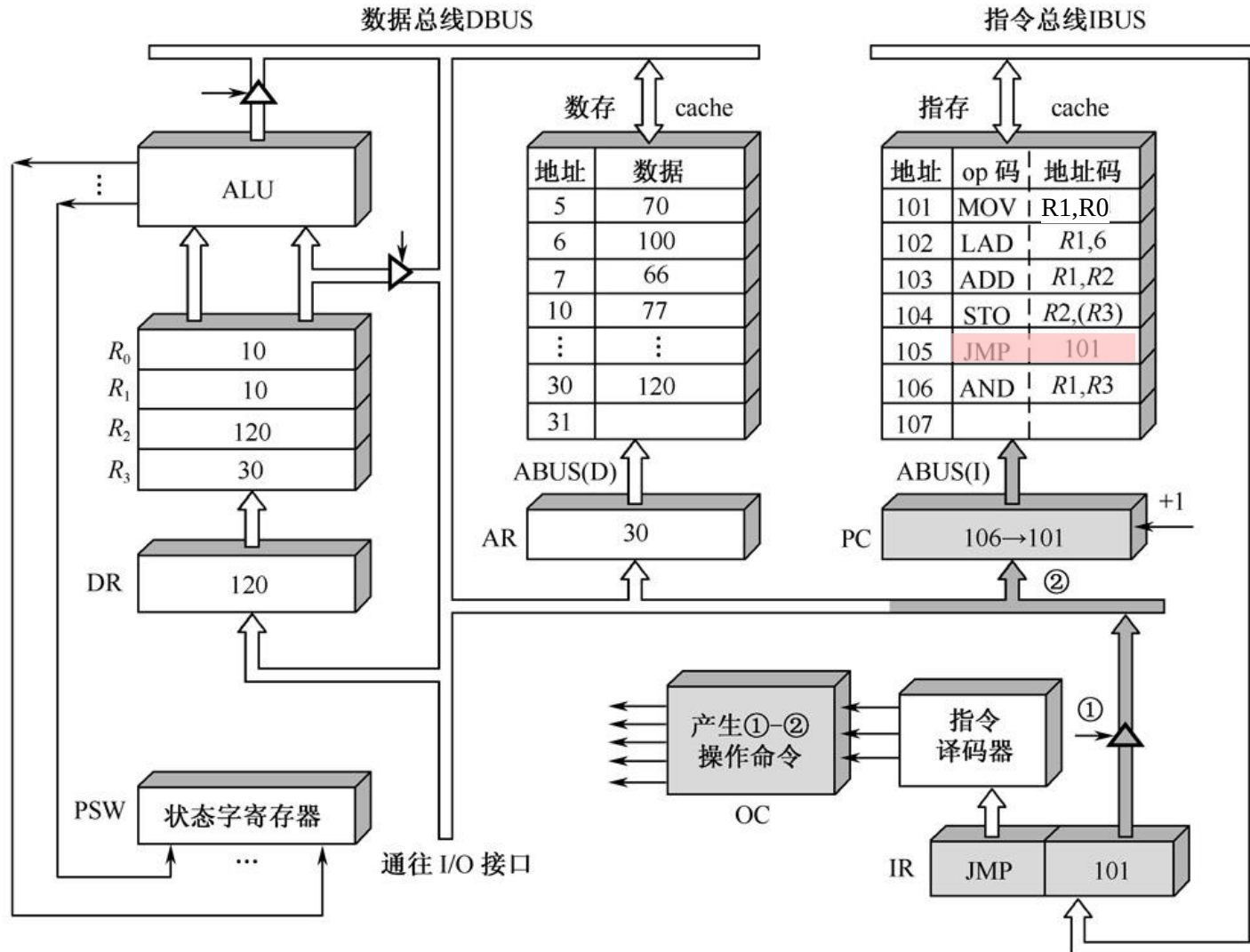


- 取指阶段
 - ◆ 从内存取出指令
 - ◆ PC+1, 为取下条指令做好准备
 - ◆ 指令译码
- 执行指令阶段
 - ◆ 完成指令所要求的操作
 - ◆ 即将指令地址码字段中的偏移值送PC





JMP指令执行周期





总结：每条指令的执行步骤

1) 读取指令

是一次读
内存操作

指令地址送入主存地址寄存器
读主存，读出内容送入指令寄存器

公共操作

2) 分析指令

3) 按指令规定内容执行指令

R_R类型指令
读写内存类型指令
输入输出类型指令
其他类型指令

不同指令的操作步骤数
和具体操作内容差异很大，
是每一条指令的特定操作
可能执行一次或多次

形成下一
条指令地址

4) 检查有无中断请求

若有，则响应中断并转中断处理
若无，则转入下一条指令的执行过程

公共操作





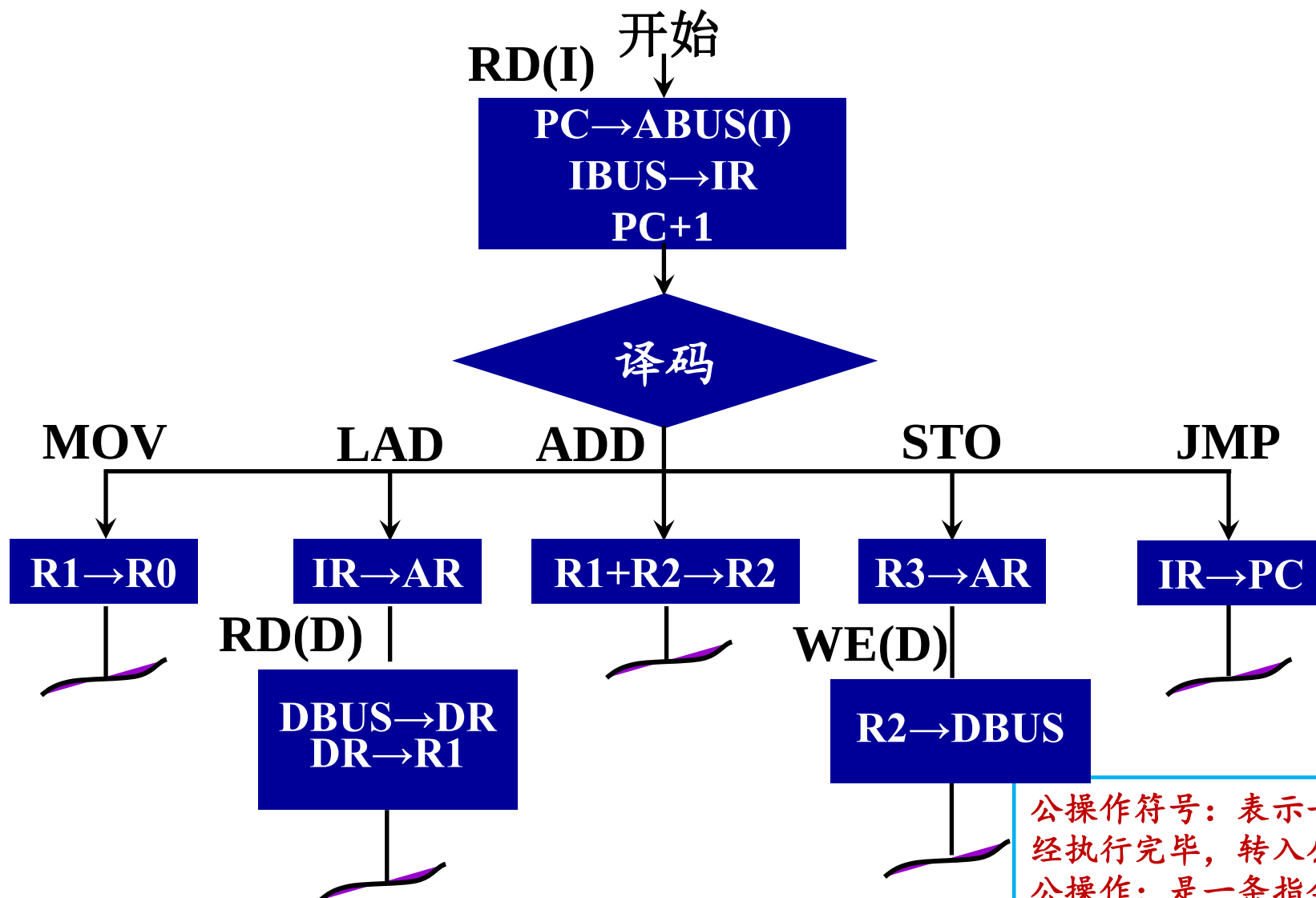
方框图语言

- 在进行中央处理器设计时，可用方框图语言来描述一条指令的指令周期
 - ◆ 方框：代表一个CPU周期，方框中的内容表示数据通路的操作或某种控制操作
 - ◆ 菱形：通常用来表示某种判别或测试，在时间上依附于紧接它的前面一个方框的CPU周期，不单独占用一个CPU周期





五条典型指令的指令周期



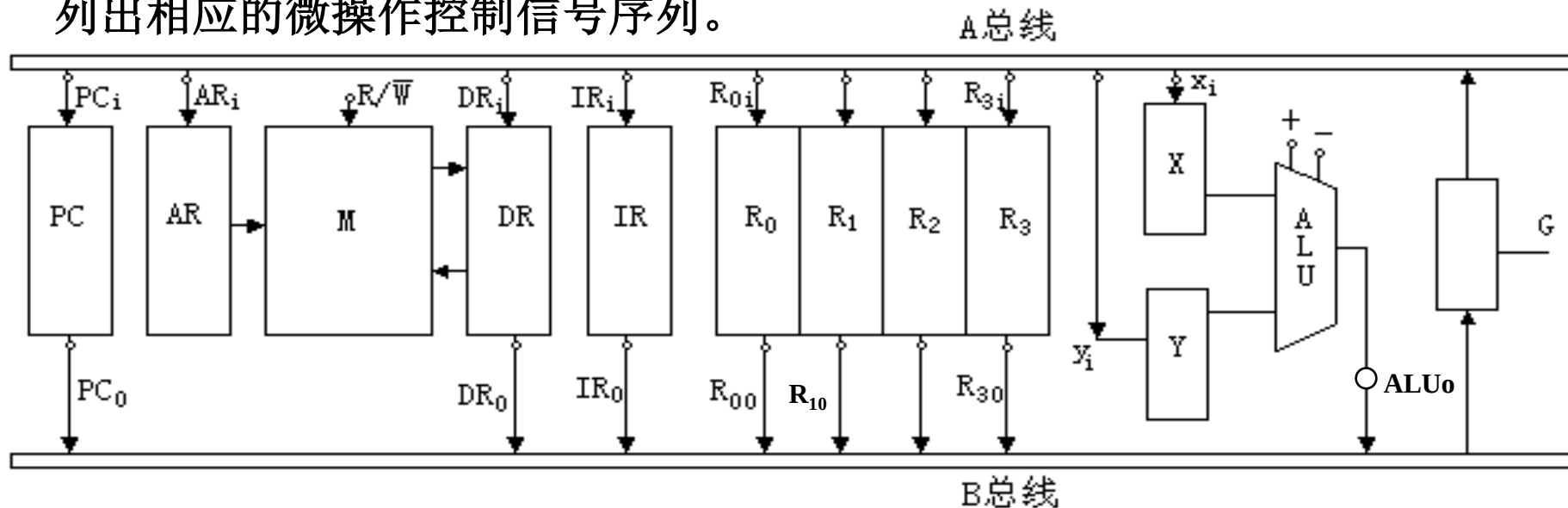
公操作符号：表示一条指令已经执行完毕，转入公操作。
公操作：是一条指令执行完毕后，CPU所进行的公共操作，如中断处理等。



例1

双总线结构机器的数据通路如图所示，IR为指令寄存器，PC为程序计数器（具有自增功能），M为主存（受R/W#信号控制），AR为地址寄存器，DR为数据缓冲寄存器，ALU由加、减控制信号决定完成何种操作，控制信号G控制的是一个门电路。另外，线上标注的小圈表示有控制信号， y_i 表示y寄存器的输入控制信号， R_{10} 为R1的输出控制信号，未标小圈的线为直通线，不受控制。

- (1)“ADD R2, R0”指令完成 $(R0)+(R2) \rightarrow R0$ 的功能，画出其指令周期流程图，假设该指令的地址已放入PC中。列出相应的微操作控制信号序列。
- (2)“SUB R1, R3”指令完成 $(R3)-(R1) \rightarrow R3$ 的功能，画出其指令期流程图，列出相应的微操作控制信号序列。

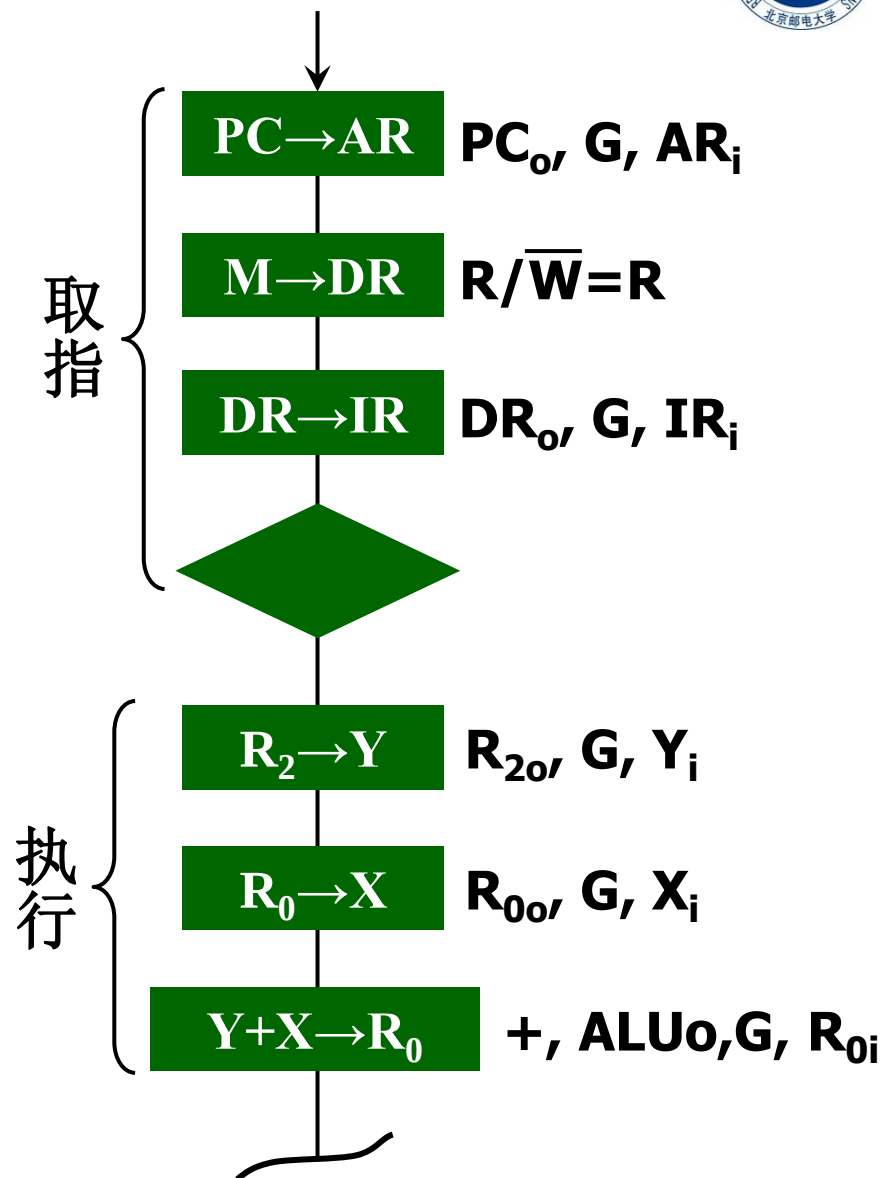




例1解 (1)

(1) “ADD R2, R0” 指令是一条加法指令，参与运算的两个数放在寄存器R2和R0中，指令周期流程图包括取指令阶段和执行指令阶段两部分。根据给定的数据通路图，

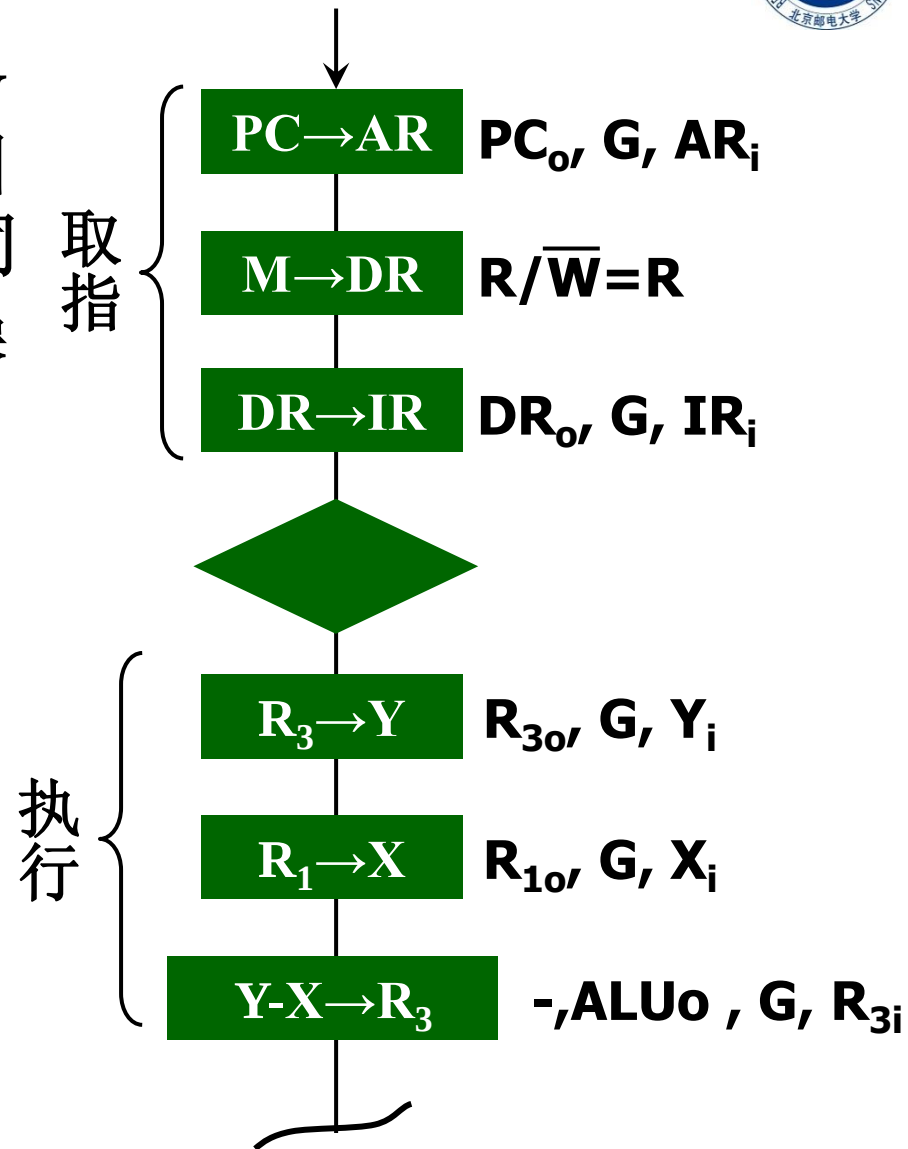
“ADD R2, R0” 指令的详细指令周期流程图如右图所示，图的右边部分标注了每一个机器周期中用到的微操作控制信号序列。





例1解 (2)

(2) “SUB R1, R3” 指令是一条减法指令，其指令周期流程图如右图所示。与ADD指令不同的是：在执行指令阶段，微操作控制信号序列有所不同。





5.3 时序产生器和控制方式



基本概念

- 时序 (Timing)：时序类似于日常生活中的“作息时间表”。计算机以时序信号为基准完成指令所规定的动作。
- 在一个CPU周期中，时间可分为若干个小段，在每一个小段，CPU完成一个特定的操作

控制器分类：

- 硬布线控制器
 - ◆ 时序信号一般采用主状态周期-节拍电位-节拍脉冲三级体制。节拍电位=CPU周期的时间
- 微程序控制器
 - ◆ 时序信号比较简单。一般采用节拍电位-节拍脉冲二级体制





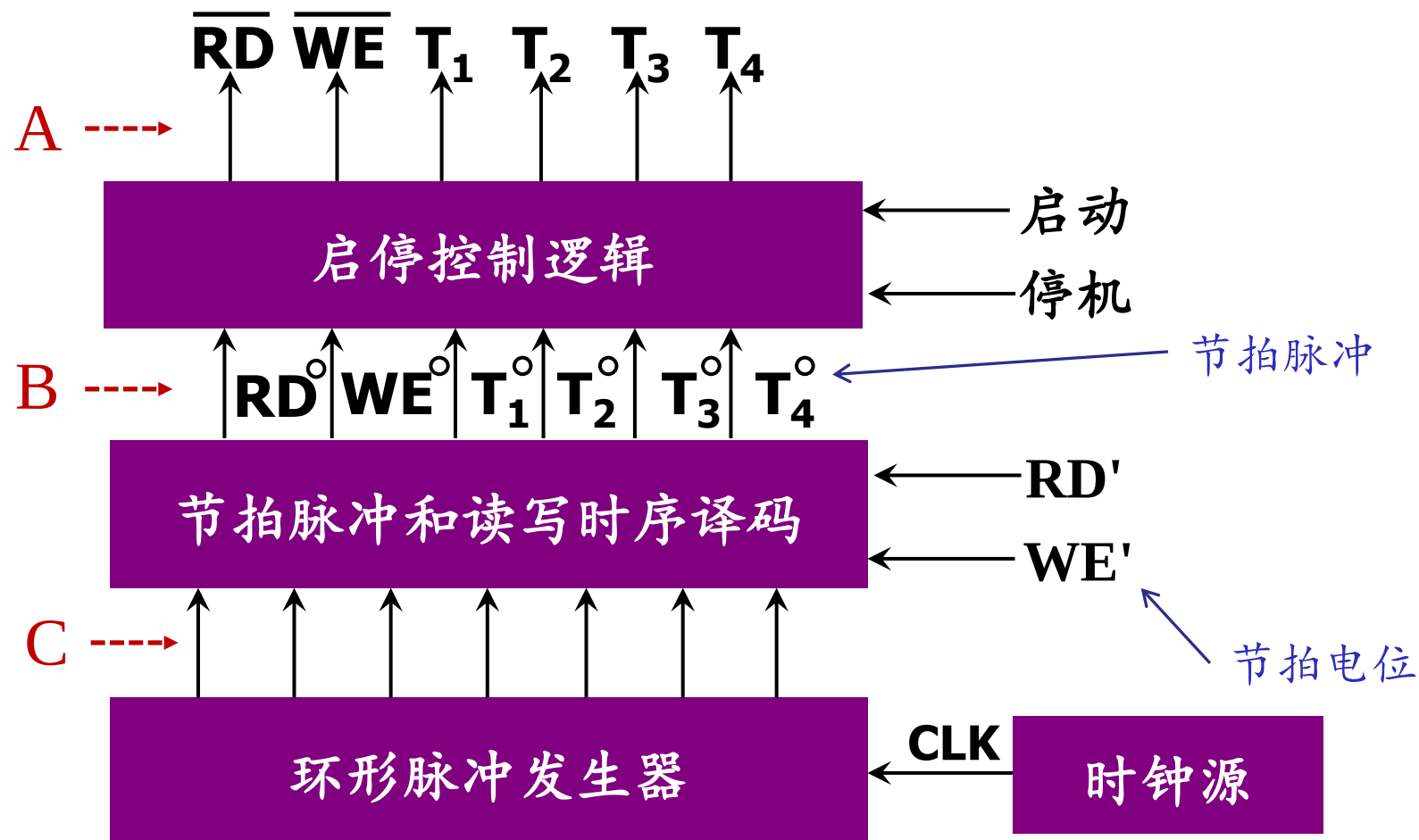
时序信号产生器

- 微程序控制器中时序信号产生器的组成
 - ◆ 时钟源
 - ◆ 环形脉冲发生器
 - ◆ 节拍脉冲和读/写时序的译码
 - ◆ 启停控制逻辑



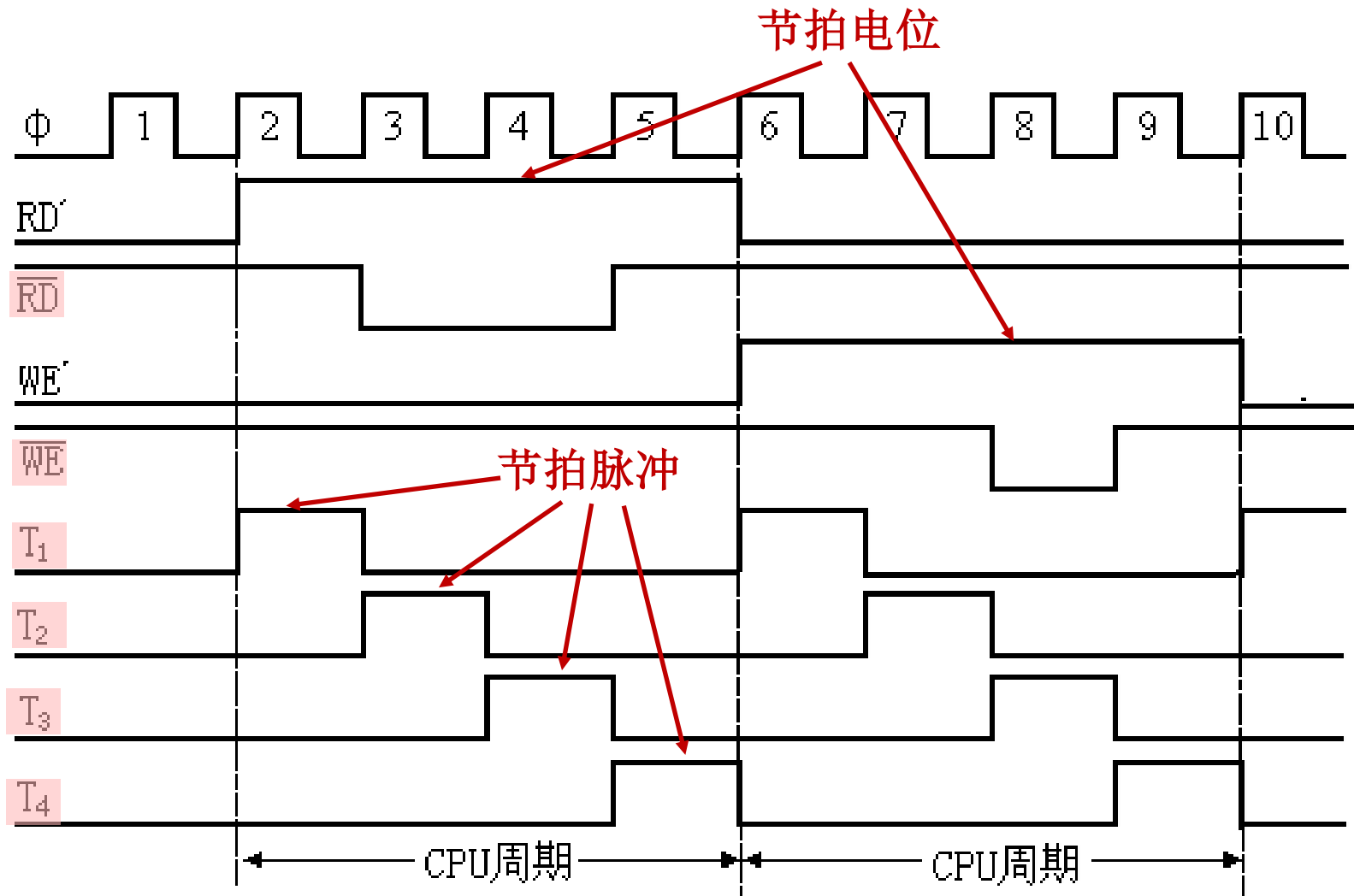


时序信号产生器框图



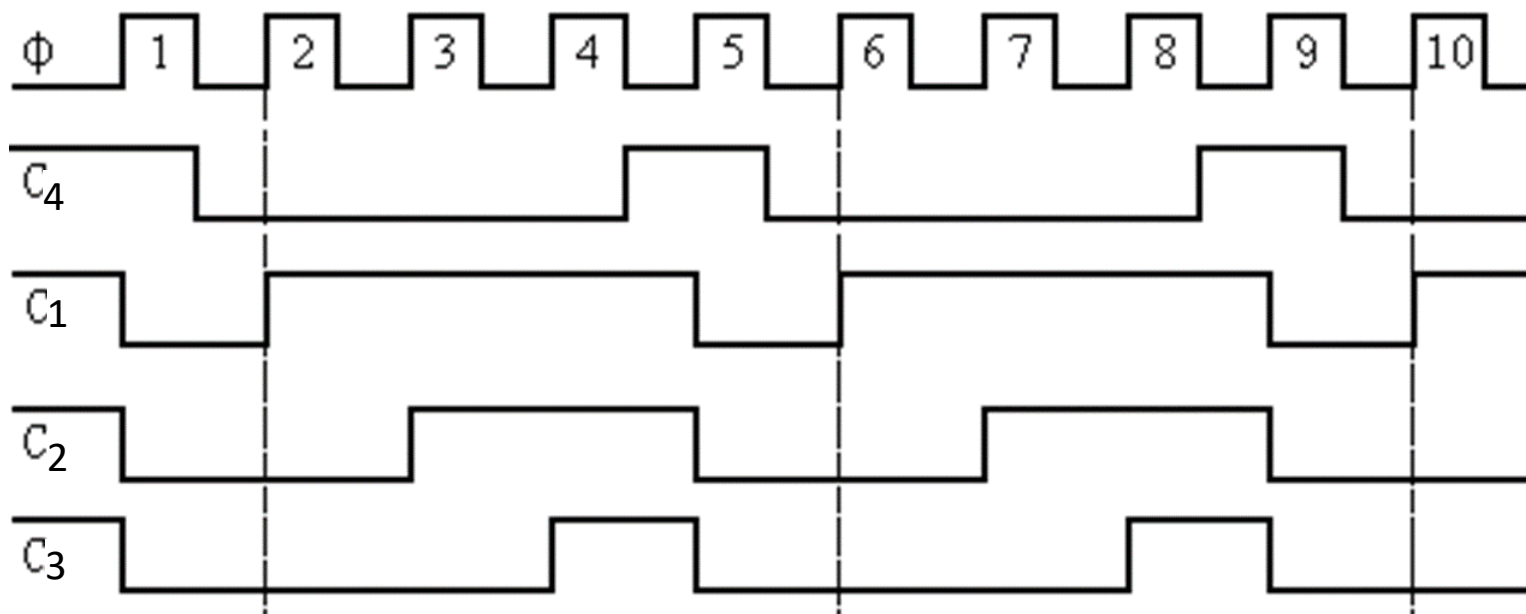


A点输出信号

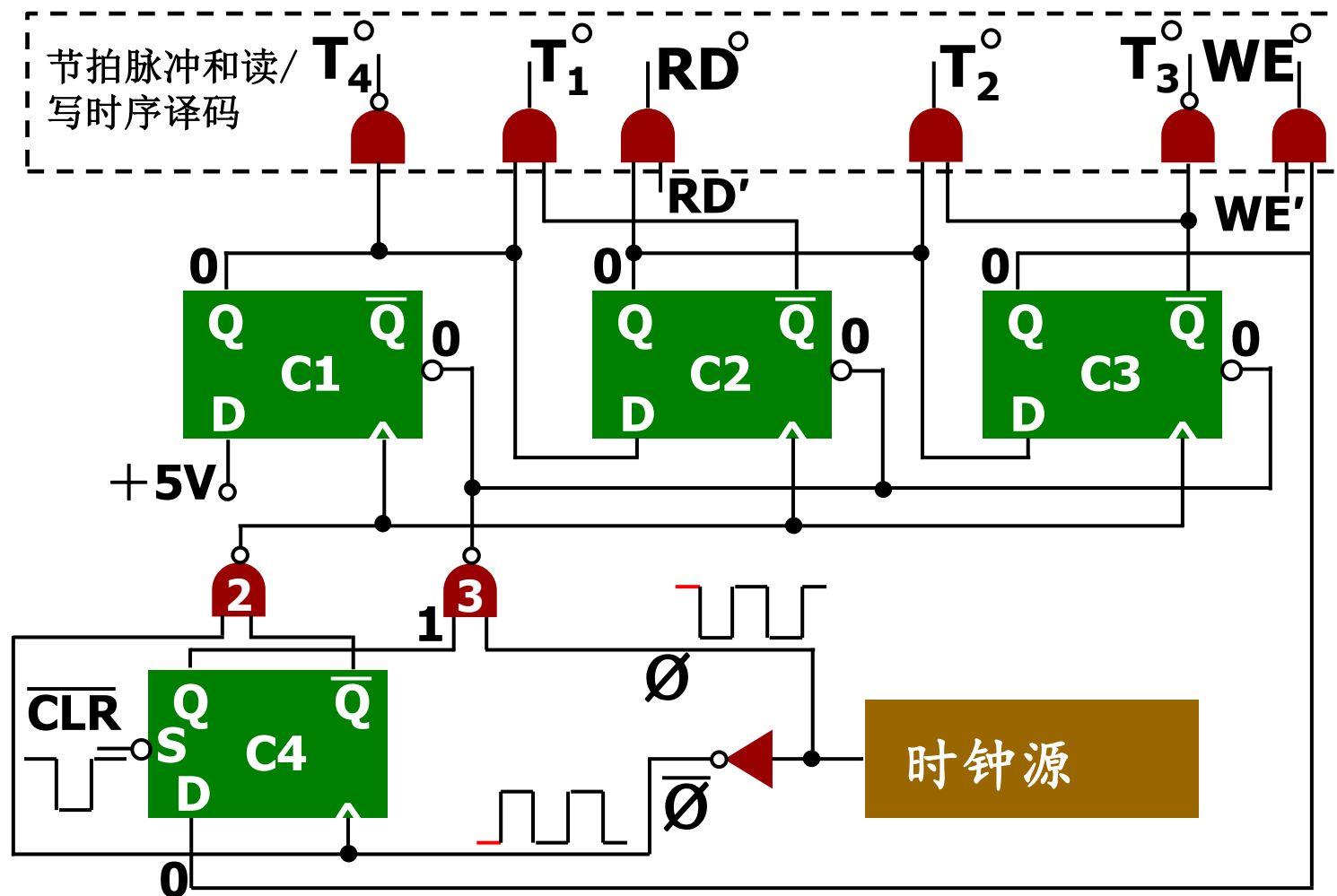


环形脉冲发生器（C点输出信号）

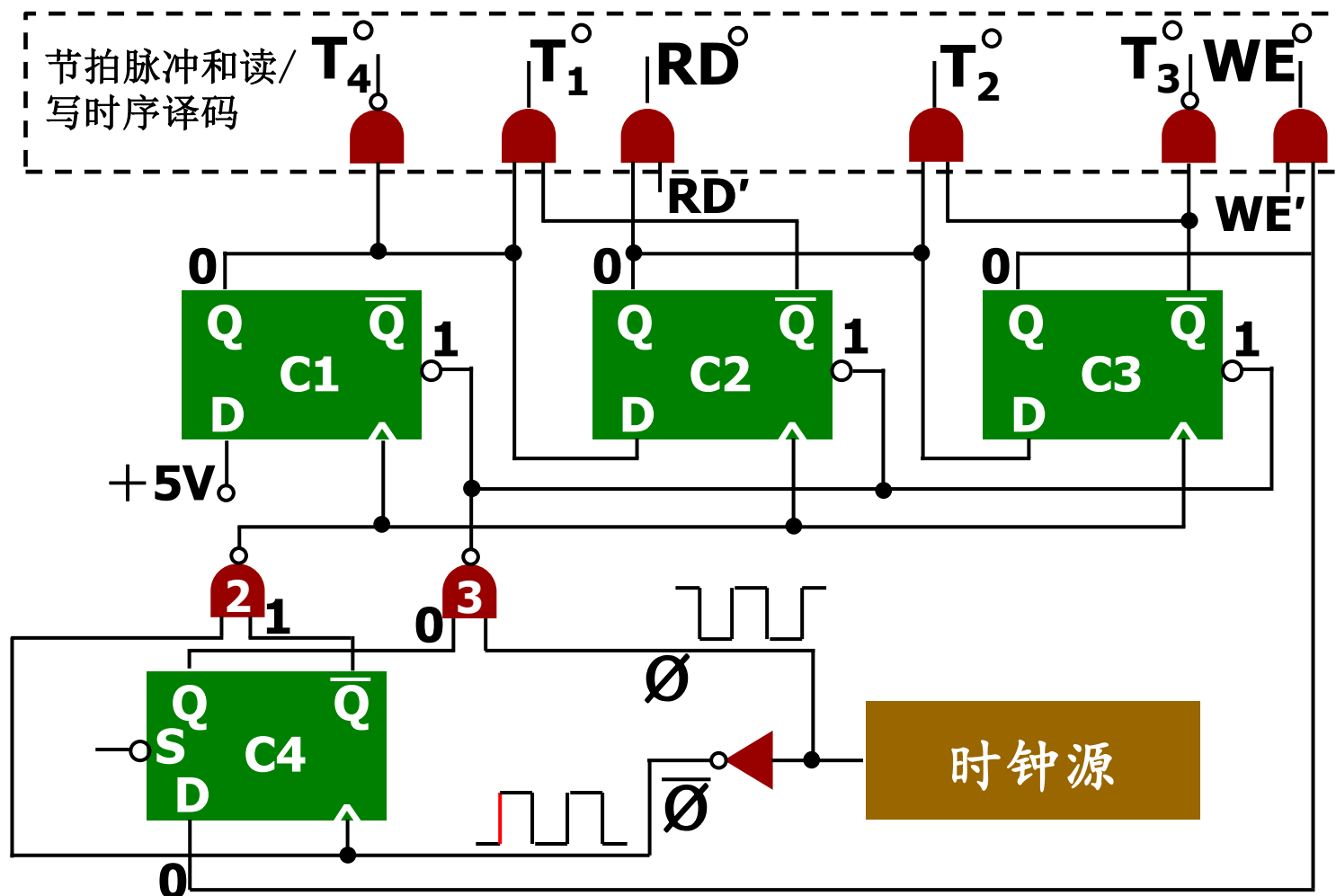
- 产生一组有序的间隔相等或不等的脉冲序列，以便通过译码电路来产生最后所需的节拍脉冲
- 为了在节拍脉冲上不帶干扰毛刺，环形脉冲发生器通常采用循环移位寄存器形式



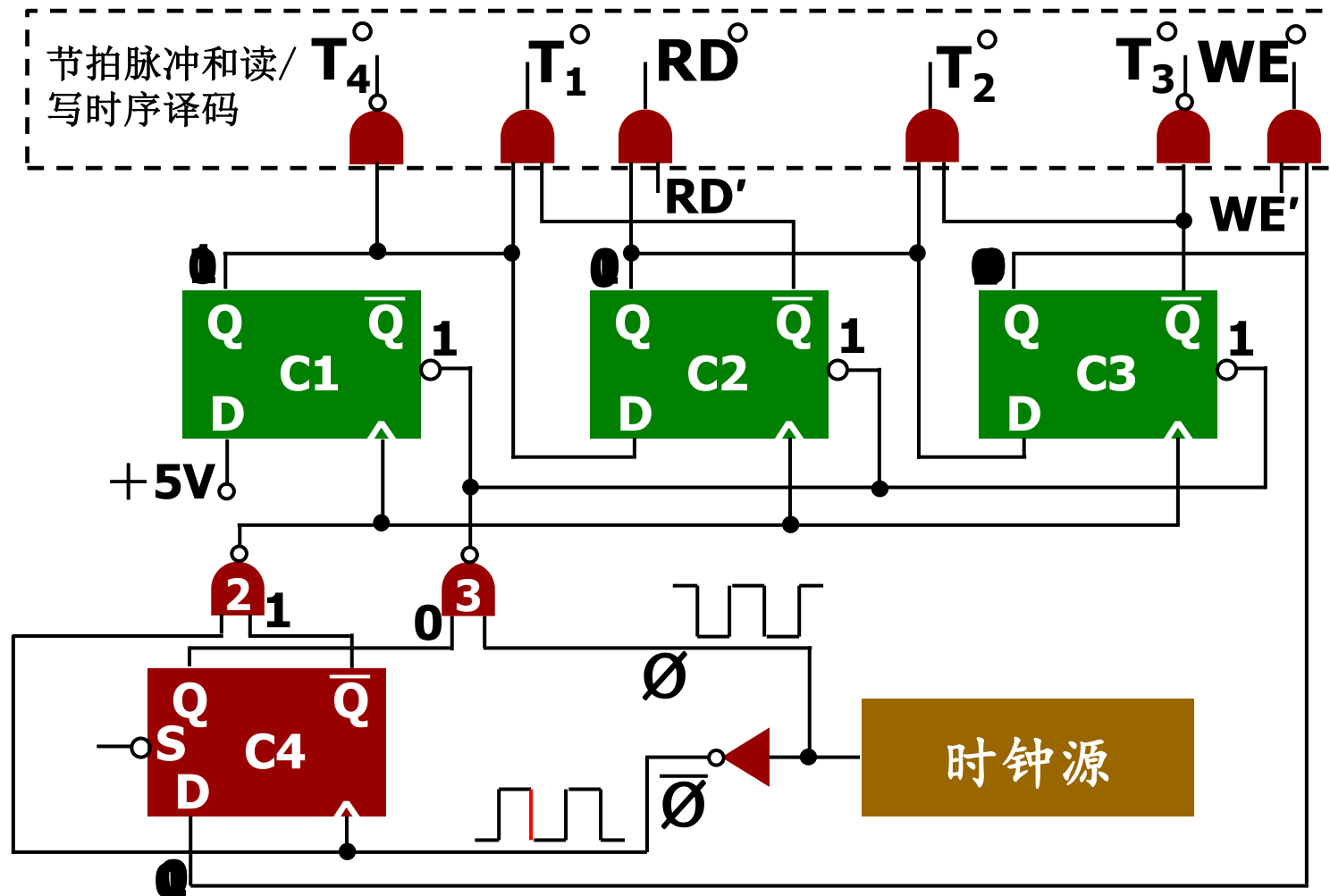
环形脉冲发生器与译码逻辑 (1)



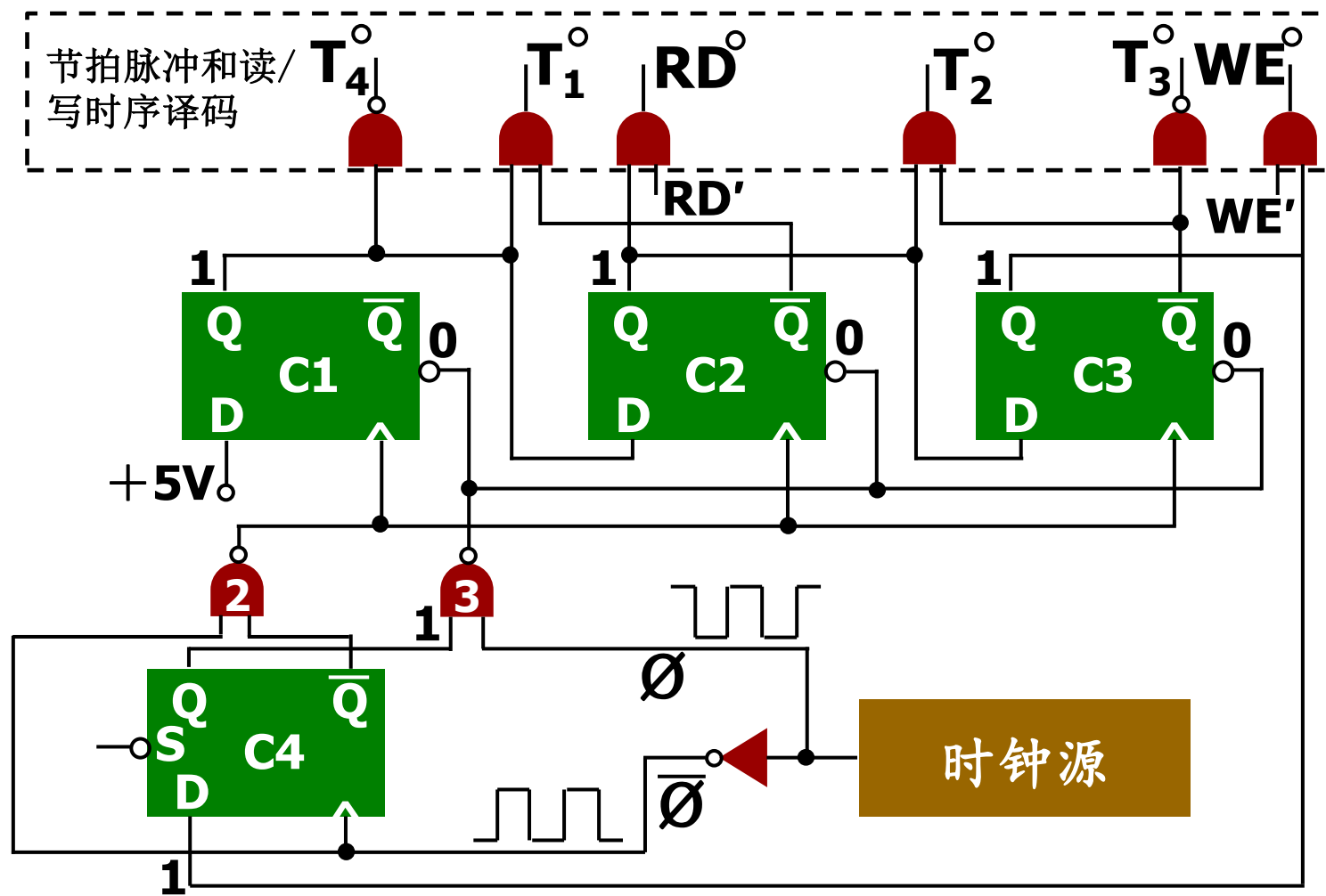
环形脉冲发生器与译码逻辑 (2)



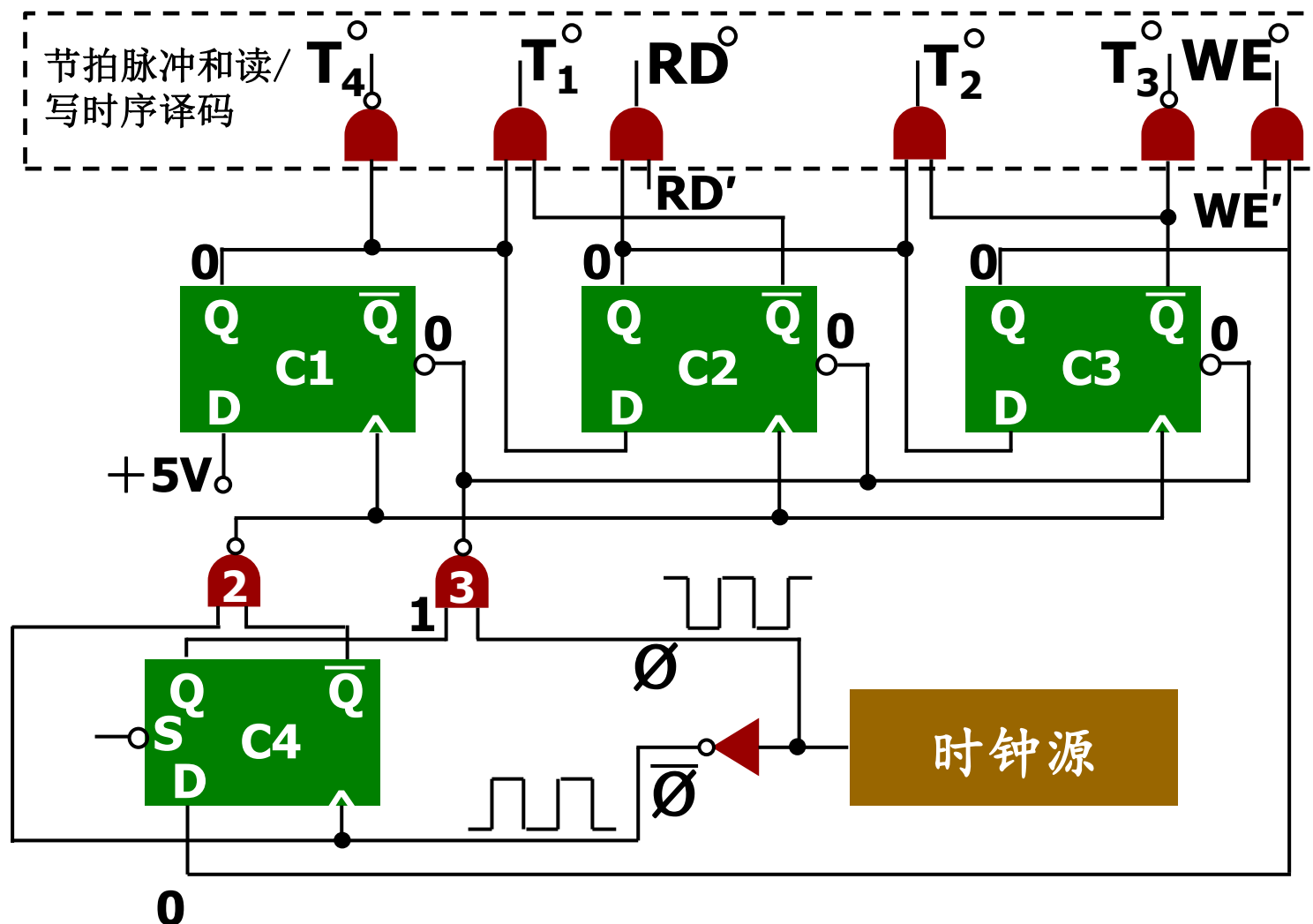
环形脉冲发生器与译码逻辑 (3)



环形脉冲发生器与译码逻辑 (4)



环形脉冲发生器与译码逻辑 (5)



$C_1 C_2 C_3: 000 \rightarrow 100 \rightarrow 110 \rightarrow 111 \rightarrow 000 \rightarrow 100$

Principles of Computer Organization



节拍脉冲和存储器读/写时序译码

- 若在一个CPU周期中产生四个等间隔的节拍脉冲，则其译码逻辑可表示为：

$$T_1^{\circ} = C_1 \cdot \overline{C_2}$$

$$T_2^{\circ} = C_2 \cdot \overline{C_3}$$

$$T_3^{\circ} = C_3$$

$$T_4^{\circ} = \overline{C_1}$$

- 存储器读/写时序信号的译码逻辑表达式为

$$RD^{\circ} = C_2 \cdot RD'$$

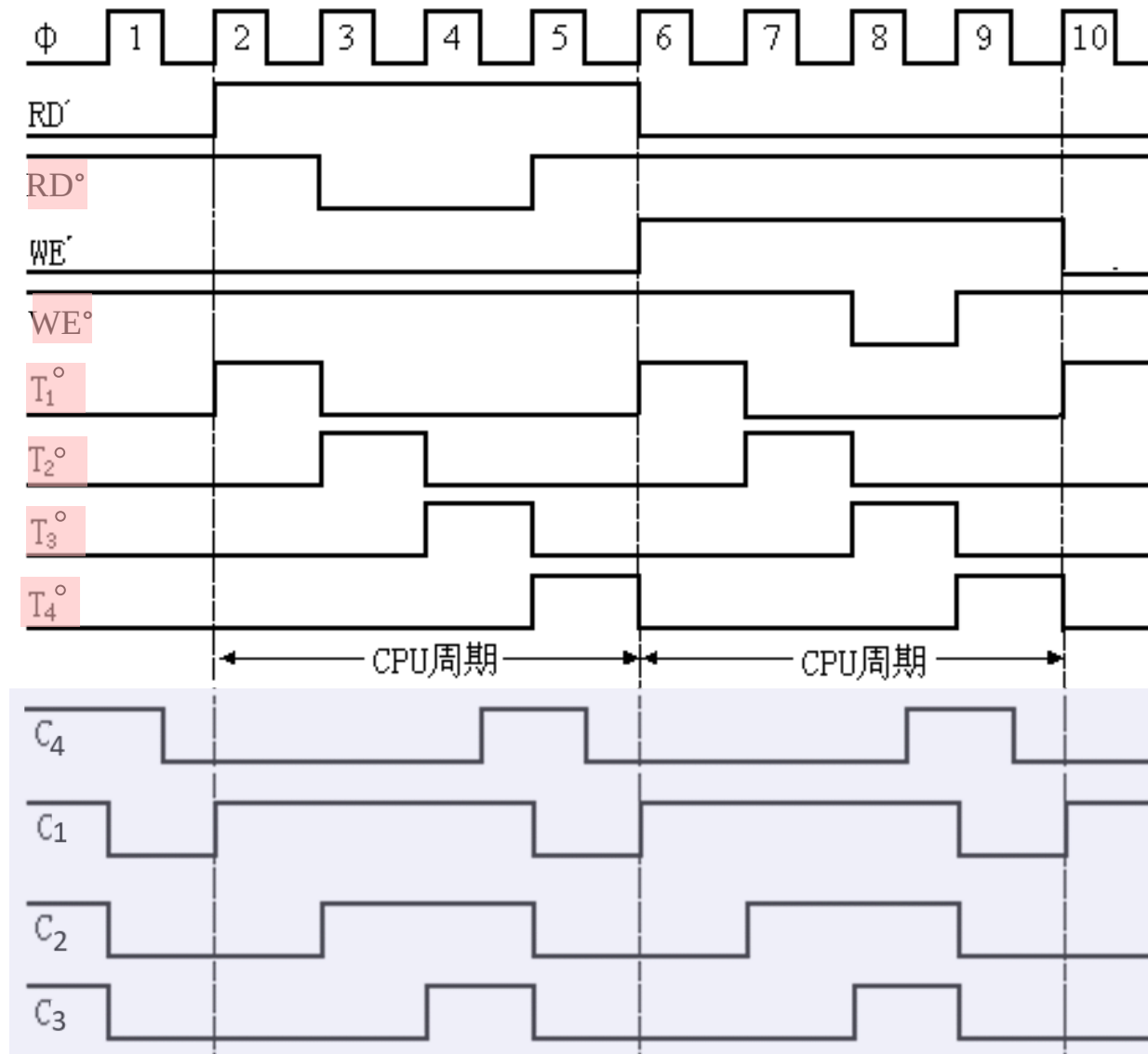
$$WE^{\circ} = C_3 \cdot WE'$$

RD' 、 WE' 信号来自微程序控制器的控制信号，是持续时间为一个CPU周期的节拍电位信号





B点输出信号





启停控制逻辑 (1)

■ 启停控制逻辑

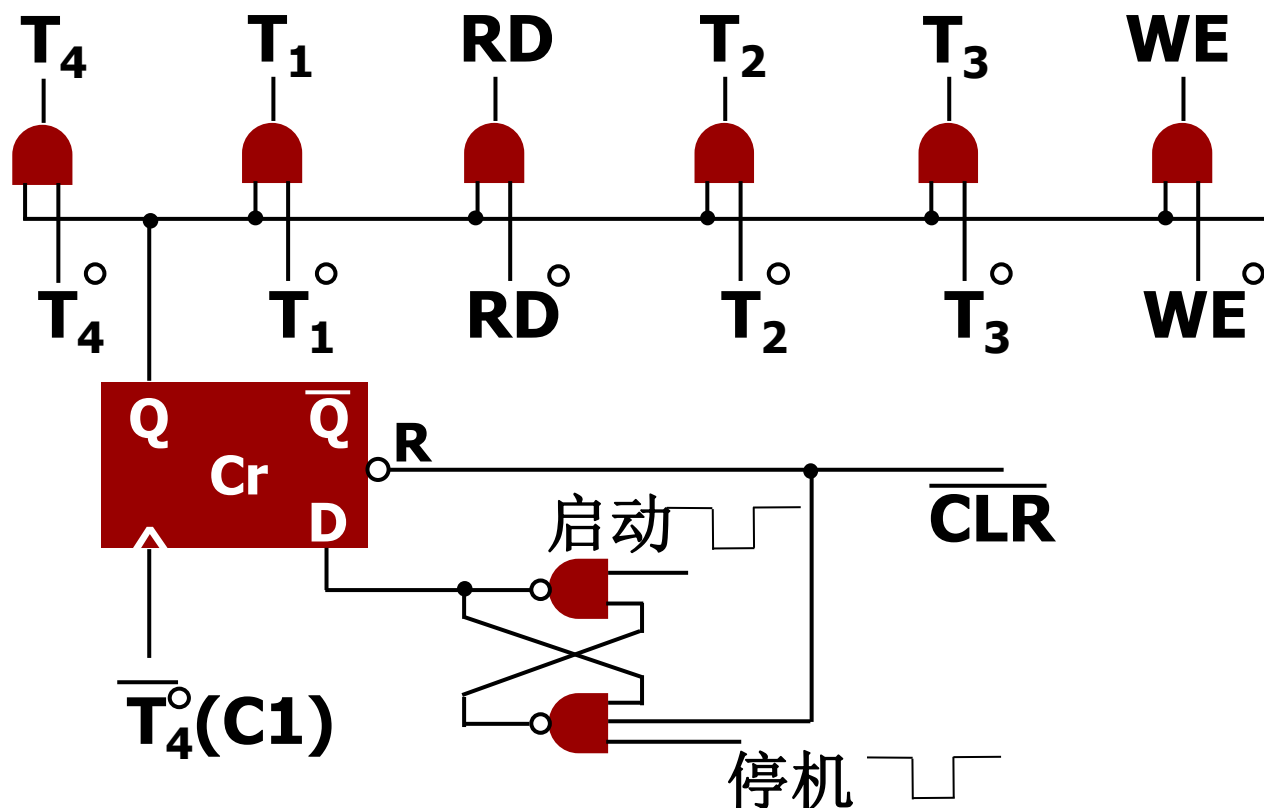
- ◆ 机器一旦接通电源，就会自动产生原始的节拍脉冲信号 $T_1^\circ - T_4^\circ$ 。但只有发出启动信号后，才允许时序产生器发出CPU工作所需的完整节拍脉冲 $T_1^\circ - T_4^\circ$
- ◆ 需要由启停控制逻辑来控制 $T_1^\circ - T_4^\circ$ 以及读/写信号的产生





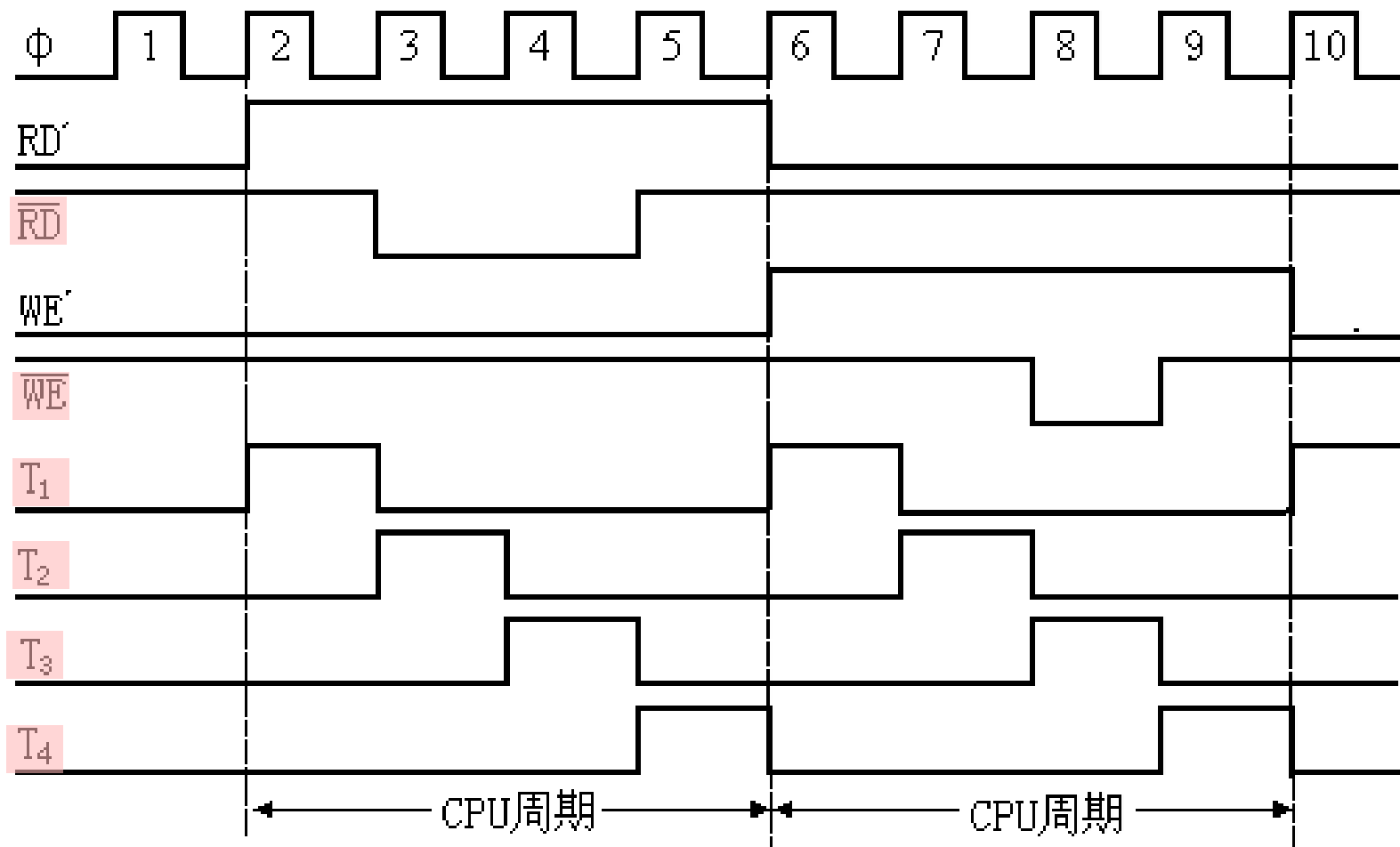
启停控制逻辑 (2)

- 当触发器Cr的Q端为“1”时，原始节拍脉冲和读/写信号通过门电路发送出去，变成CPU需要的节拍脉冲信号和读写时序；Q端为“0”时，关闭时序产生器





启停信号保证产生完整的CPU周期信号





控制方式

■ 控制器的控制方式

- ◆ 控制不同操作序列时序信号的方法

■ 常用的控制方式

- ◆ 同步控制
- ◆ 异步控制
- ◆ 联合控制





1、同步控制方式

- 在任何情况下，各指令在执行时所需的机器周期数和时钟周期数都固定不变
- 同步控制方式的三种方案如下：
 1. 采用完全统一的机器周期执行各种不同的指令。简单指令会导致时间浪费
 2. 采用不定长机器周期。复杂指令采取延长机器周期的方法
 3. 中央控制与局部控制结合。将大部分指令安排在固定的机器周期完成，称为中央控制，对少数复杂指令（乘、除、浮点运算）采用另外的时间进行，称为局部控制。





2、异步控制方式

- 每条指令、每个操作控制信号**按需**占用时间
- 每条指令的指令周期可由多个不等长的机器周期构成
- 指令的操作控制序列通常不依赖时钟周期信号，而是采用控制器发出“请求”信号、执行部件完成操作后回复“应答”信号的方式，





3、联合控制方式

同步控制和异步控制相结合的方式，两种方法：

1. 大部分操作序列安排在固定长度的机器周期中，对某些时间难以确定的操作则以执行部件的“应答”信号作为本次操作的结束
2. 各条指令的机器周期数不相同，但机器周期的节拍脉冲数相同





5.4 微程序控制器



5.4.1 微程序控制原理





基本概念（1）

- 1951年，在曼彻斯特大学召开的计算机会议上，英国剑桥大学的Maurice Vincent Wilkes教授首先提出了微程序的概念和原理
- 微程序设计技术是利用软件方法来设计硬件的一种技术
 - ◆ 微程序控制器与硬布线控制器相比较，具有规整性、灵活性、可维护性等一系列优点





基本概念（2）

■ 微程序控制的基本思想

- ◆ 用多条微指令（即：一个微程序）解释每条指令的执行过程。全部的微程序有机地组合在一起，存储在**控制存储器**（只读存储器）中
- ◆ 当机器运行时，一条一条地读出这些微指令，从而产生全机所需要的各种操作控制信号，使相应的部件执行特定的操作





微命令和微操作

■ 计算机的构成

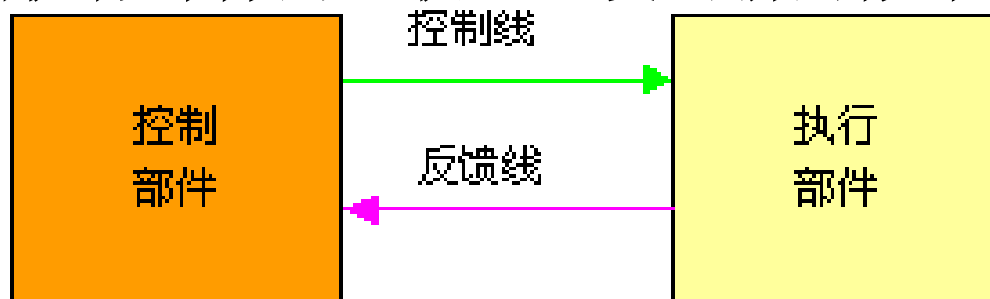
- ◆ 控制部件—控制器
- ◆ 执行部件—运算器、存储器、外围设备

■ 微命令—控制部件通过控制线向执行部件发出各种控制命令（即：控制信号）

■ 微操作—执行部件接受微命令后所进行的特定操作

■ 反馈信息

- ◆ 通常执行部件使用反馈线向控制部件报告操作情况，控制部件则根据执行部件的“状态”发出新的微命令





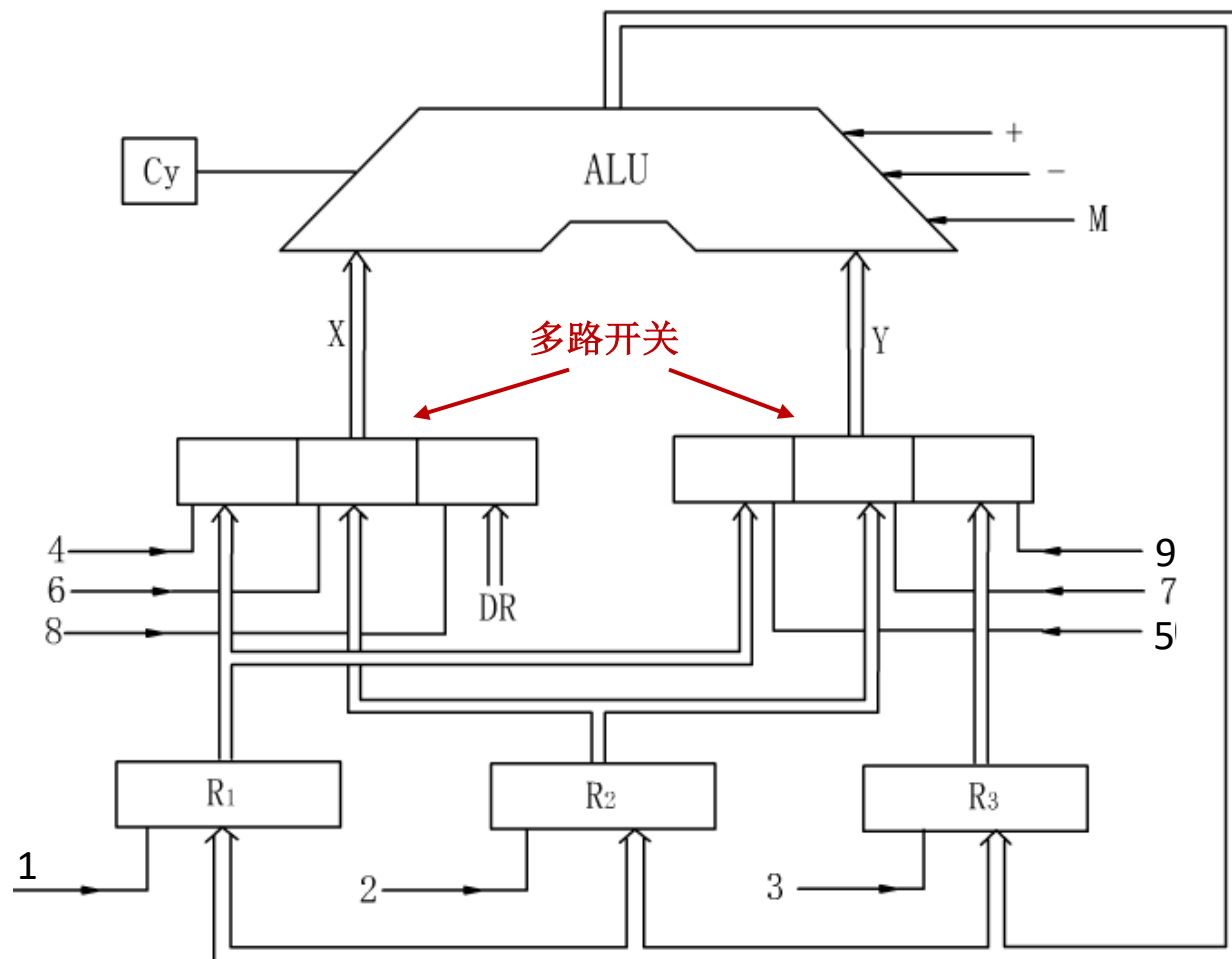
微操作分类

- 相容性操作：同时或在同一个CPU周期内可以并行执行的微操作
- 相斥性操作：不能同时或不能在同一个CPU周期内并行执行的微操作



简单运算器数据通路图

- “+”、“-”、“M”三个微操作是相斥性微操作
- 4/6/8三个微操作是相斥的，5/7/9也是相斥性的
- 1/2/3可同时进行的，所以是相容性的微操作
- ALU的X输入微操作4/6/8和Y输入的5/7/9任意两个微操作，也都是相容性的





微指令

- 在机器的一个CPU周期中，实现某个操作功能的一组微命令就构成一条微指令

或者说

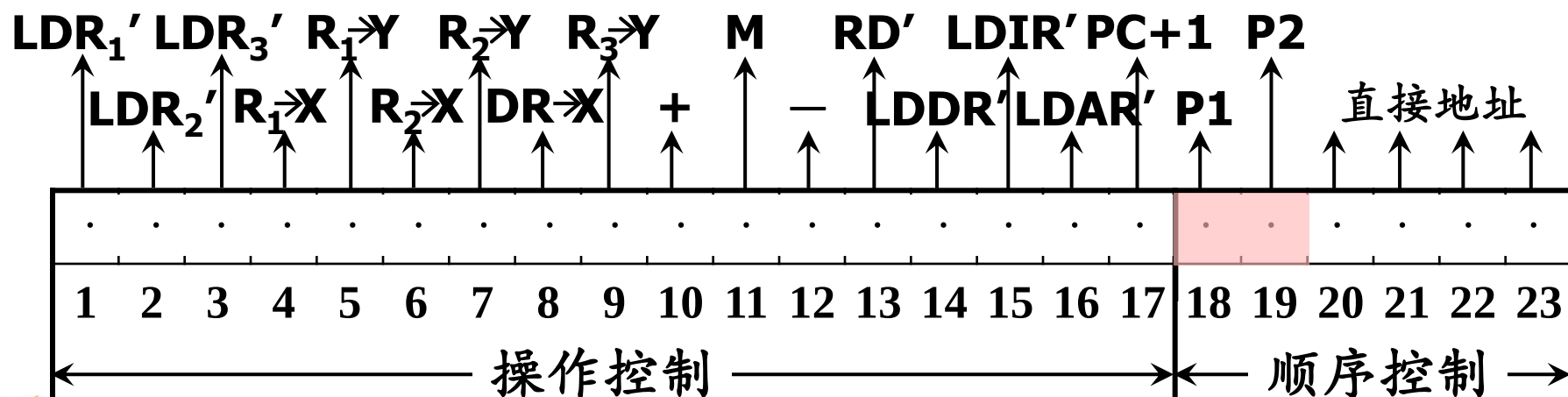
- 用一条微指令对应一条机器指令的一个执行步骤
- 微指令需要具备的2个功能
 - ◆ 提供一条机器指令的一个执行步骤所需要的控制信号，以实现该执行步骤的操作功能
 - ◆ 提供下一条待执行微指令的地址，以便自动有序地读出每一条微指令，解决机器指令执行步骤之间的正确的接续关系



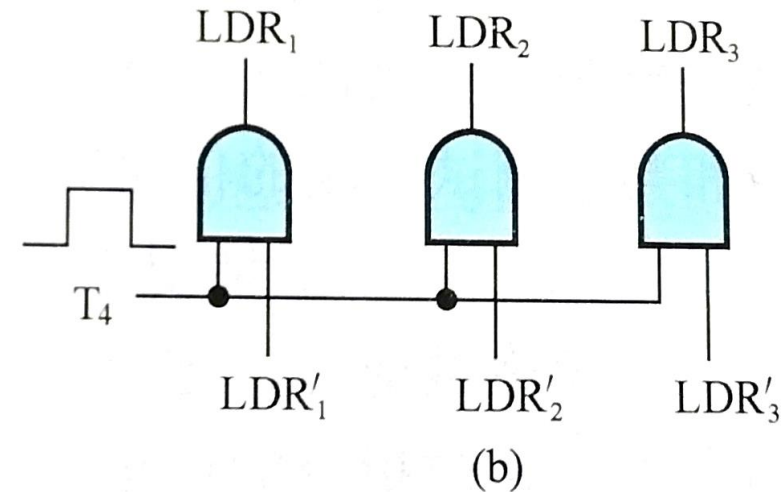
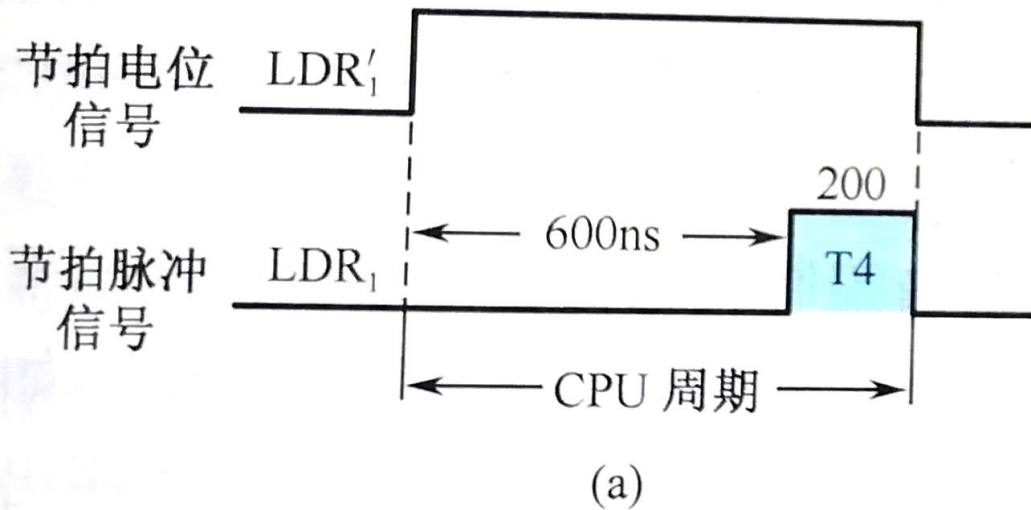


微指令基本格式

- 微指令由操作控制和顺序控制两大部分组成，下图中微指令字长为23位，
- 操作控制部分用来发出控制信号，每一位表示一个微命令
 - ◆ 控制信号均为节拍电位信号，持续时间都是一个CPU周期，只有与节拍脉冲相结合才能得到实际控制信号
- 顺序控制部分用来指明下一条微指令的地址
 - ◆ 微指令中最后4位直接给出下一条微指令的地址，另外两位为测试方式位，根据测试结果来决定下一条微指令的地址



运算器操作时序与产生逻辑





微程序控制器

■ 微程序控制器

- ◆ 控制存储器

- ◆ 微指令寄存器

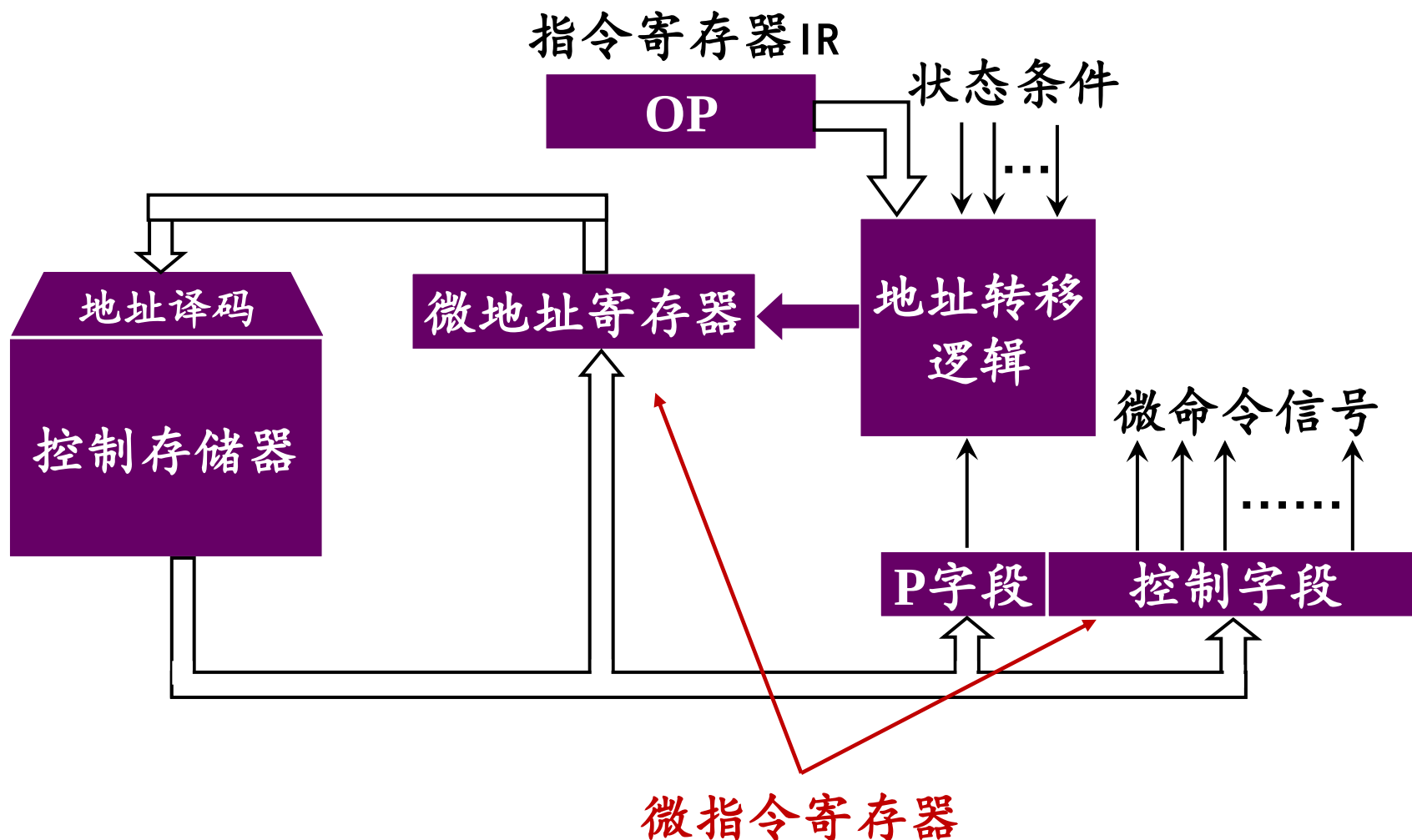
 - 微地址寄存器

 - 微命令寄存器

- ◆ 地址转移逻辑



微程序控制器原理框图 (1)





微程序控制器原理框图（2）

■ 控制存储器

- ◆ 用来存放实现全部指令系统的微程序的只读存储器
- ◆ 读出一条微指令并执行的时间总和称为一个微指令周期

■ 微指令寄存器

- ◆ 用来存放由控制存储器读出的一条微指令
- ◆ 其中微地址寄存器决定将要访问的下一条微指令的地址

■ 地址转移逻辑

- ◆ 一般情况下，由微指令的顺序控制部分直接给出下一条微指令的地址（微地址），存放在微地址寄存器中
- ◆ 微程序出现分支时，地址转移逻辑完成修改微地址的功能





控制存储器

000	微命令	P	001	}	取指微程序
001	微命令	p	002		
002	微命令	P=1	002		
	...				
xxx	微命令	P	xxx+1	}	ADD操作微程序
	微命令	p	xxx+2		
	微命令	p	000		
	...				
yyy	微命令	P	yyy+1	}	LDA操作微程序
	微命令	p	yyy+2		
	微命令	p	000		
	...				



微程序特点

- 一条机器指令是若干条微指令组成的序列来实现的
- 一条机器指令对应着一段微程序，而微程序的总和便可实现整个的指令系统
- 微程序设计可以很容易地在不同的微体系结构上实现相同的指令系统

不同的体系结构



如：x86-64，ARMv8，MIPS，RISC-V

不同的微体系结构



如：x86-64的Cypress Cove，Golden Cove



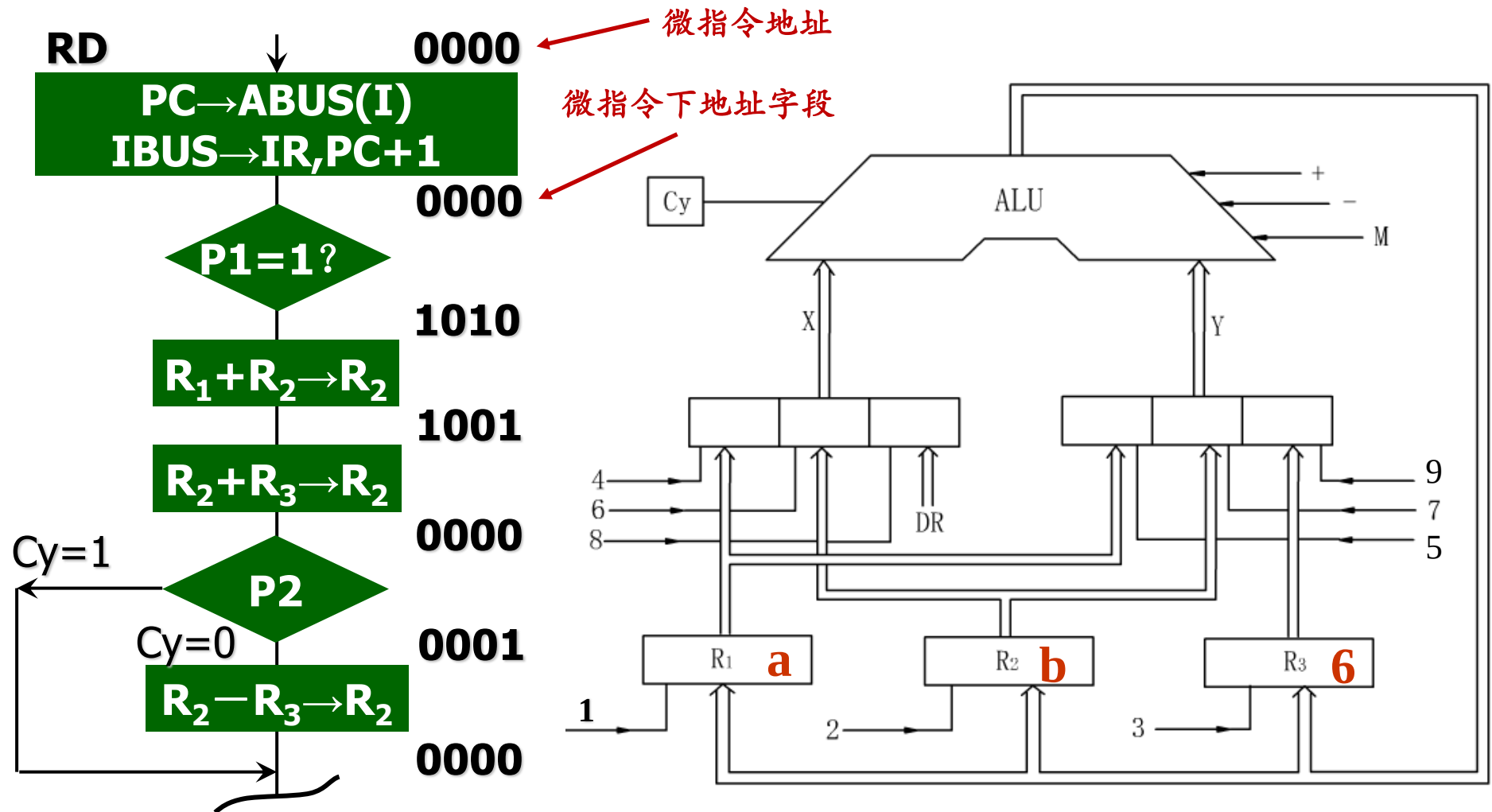
微程序举例（1）

- 十进制加法
 - ◆ 用BCD码来完成十进制数的加法运算
 - ◆ 当二数相加之和大于9时，必须对和数进行加6修正
- 假设数a和b已存放在R1和R2寄存器中，数6存放在寄存器R3中
- 算法思路：先进行 $a+b+6$ 运算，然后判断结果有无进位：当进位标志 $Cy=0$ ，减去6；当 $Cy=1$ ，不减6
- 画出十进制加法微程序流程图，由四条微指令组成
 - ◆ 第一条是“取指”微指令
 - ◆ 第二条微指令完成 $a+b$ 运算
 - ◆ 第三条微指令完成 $a+b+6$ 运算，同时进行判别测试
 - ◆ 第四条微指令完成和数减6运算





微程序举例 (2)





微程序举例（3）

■ 微程序的调用

- ◆ 用取出的十进制加法指令操作码作为偏移量，通过微地址映射部件查表得到微程序在控制存储器中的地址，就可以调出所需要的微程序（如上图中微程序地址为1010）
- ◆ 微地址映射部件是用**ROM**实现的，地址输入为指令寄存器**IR**的操作码，输出为该指令对应的微程序段的入口地址



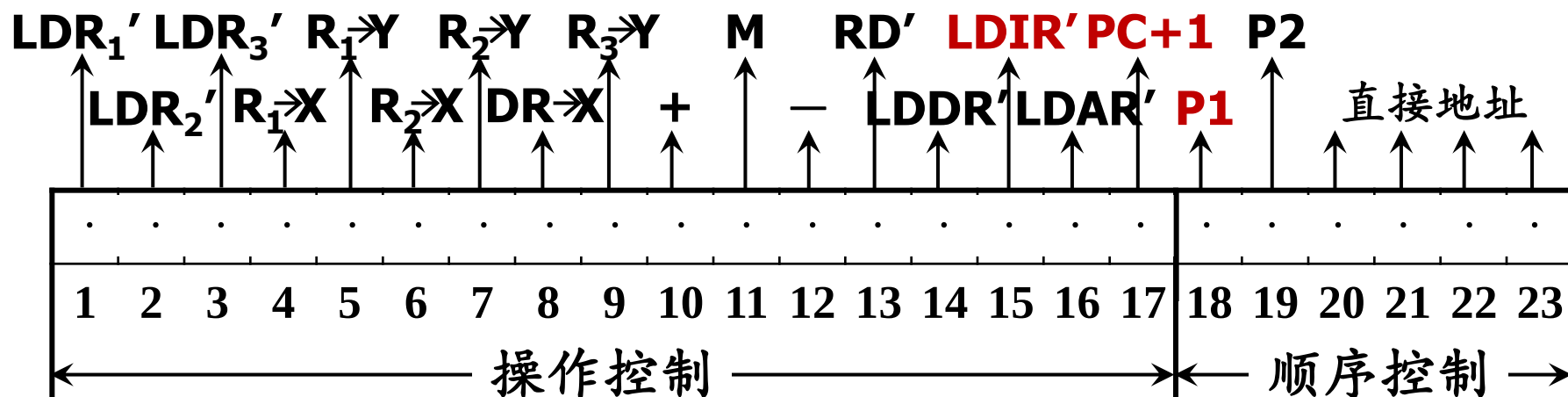


微程序举例 (4)

■ 第一条微指令的二进制编码是

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

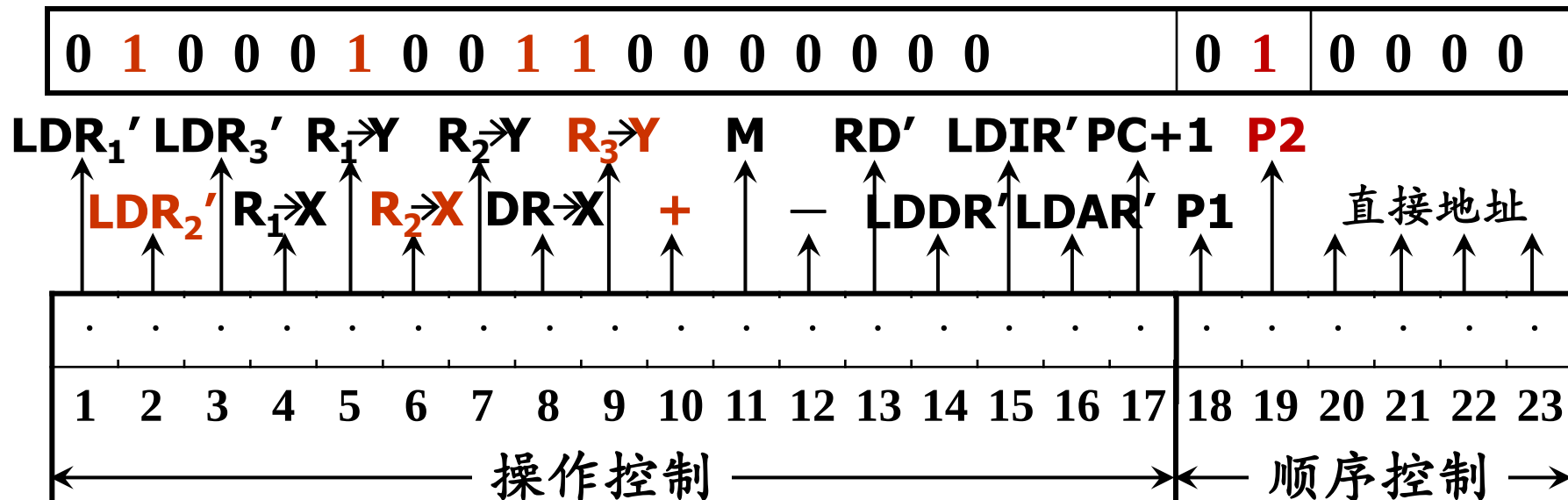
- ◆ 操作控制字段有两个微命令：**LDIR'**发出指存读命令**RD(I)**，指令放到**IBUS**上，然后发出打入**IR**命令；发出**PC+1**命令
- ◆ 顺序控制字段指明下一条微指令的地址是0000，但由于第18位**P1=1**，要求用**OP**字段查表形成下一条微指令地址1010





微程序举例 (6)

- 在第三个CPU周期，按照1001这个微地址读出第三条微指令，其二进制编码是



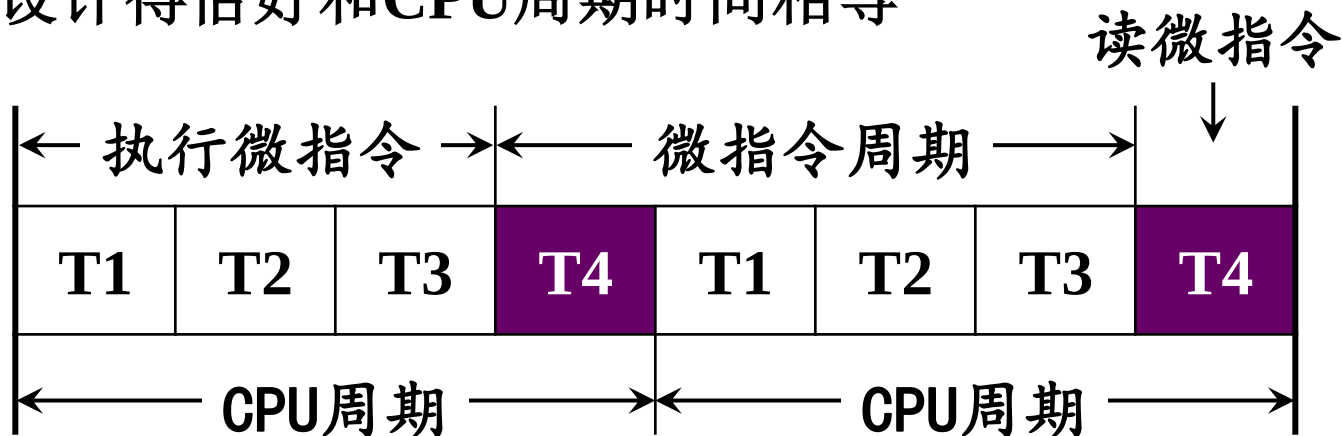
- 这条微指令中，操作控制部分发出四个微命令，完成 $R2+R3→R2$ 运算
- 顺序控制部分的测试字段P2为1，表示需要进行P2测试，P2测试就是检测标志位Cy，当Cy=0时，下一条微指令的地址为0001；而当Cy=1时，下一条微指令的地址为0000





微指令周期与CPU周期的关系

- 在串行方式的微程序控制器中，**微指令周期**等于读出微指令的时间加上执行该条微指令的时间
- 为了保证整个机器控制信号的同步，可以将一个微指令周期时间设计得恰好和CPU周期时间相等



- 一个CPU周期为 $0.8\mu\text{s}$ ，包含四个等间隔的节拍脉冲T1~T4，每个脉冲宽度为 200ns
- 用T4作为读取微指令的时间，用T1+T2+T3时间作为执行微指令的时间（T4上升沿将运算结果打入某寄存器，同时读取下一条微指令）





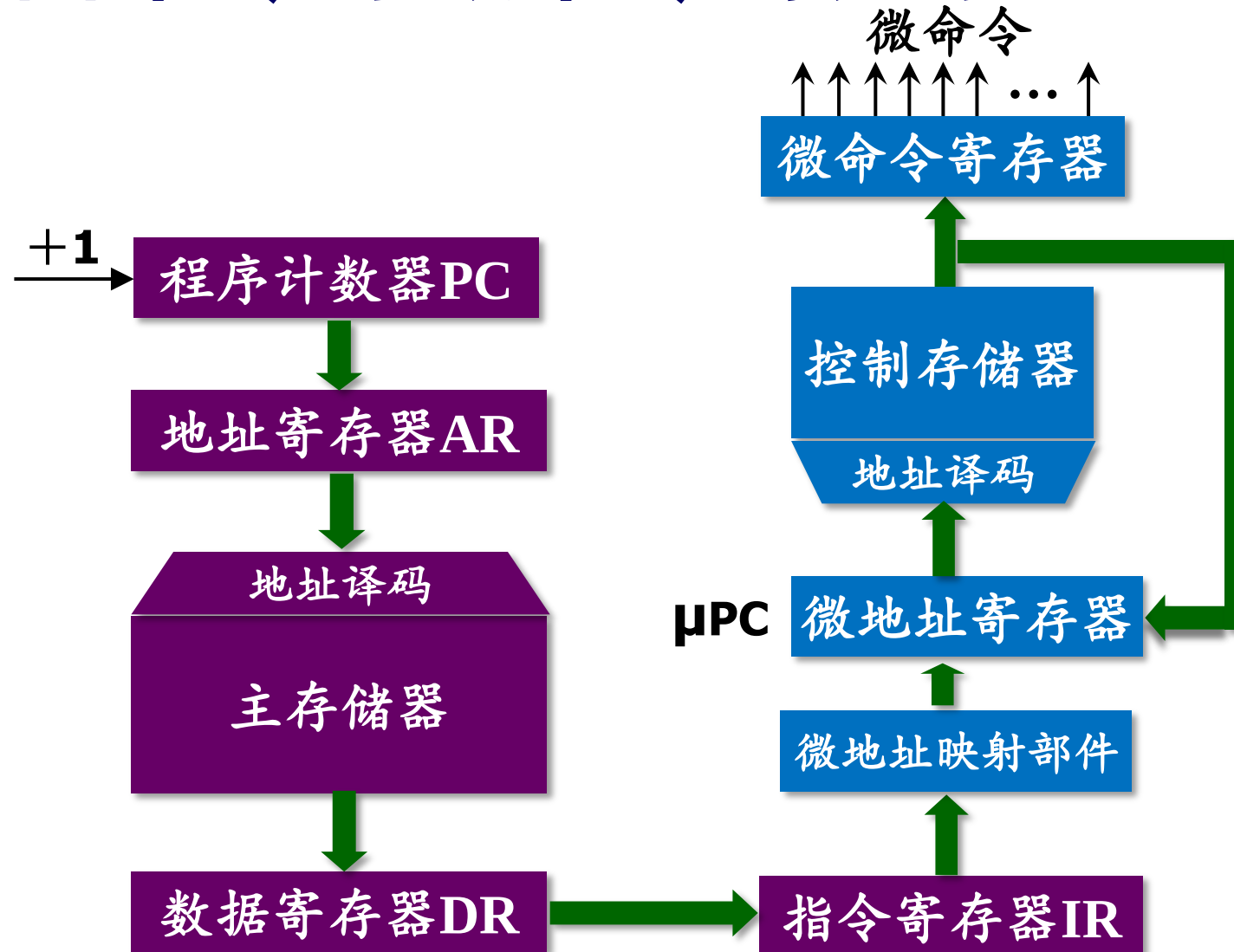
机器指令与微指令的关系（1）

- 一条机器指令对应一个微程序，这个微程序是由若干条微指令序列组成的
- 从指令与微指令，程序与微程序，地址与微地址的对应关系来看，前者与主存储器有关，后者与控制存储器有关





机器指令与微指令的关系 (2)





5.4.2 微程序设计技术





1. 微命令编码

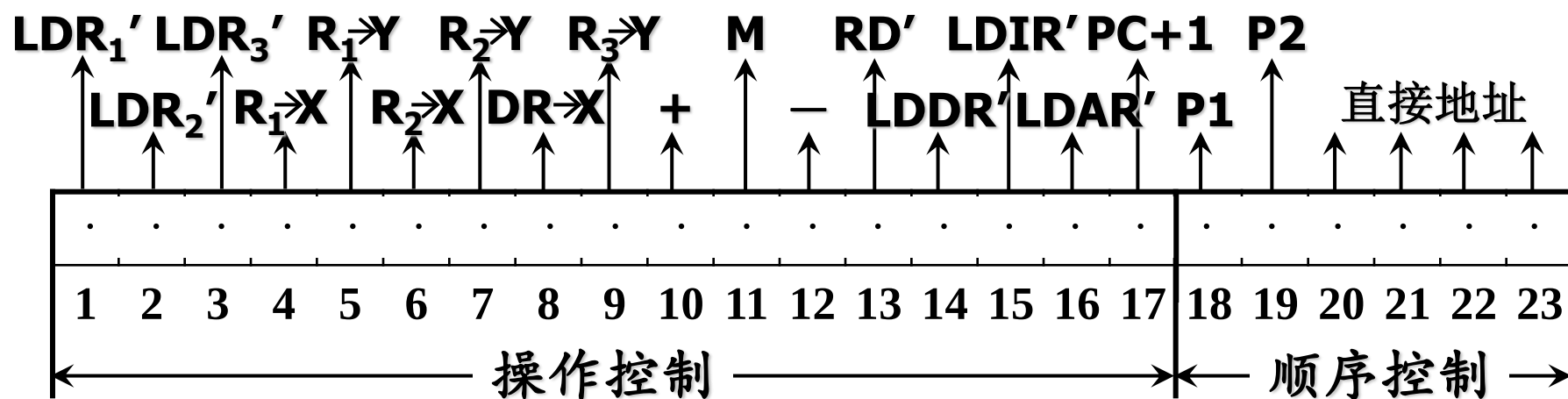
- 微指令操作控制字段的微命令表示方法可分为3类
 - ◆ 直接表示法
 - ◆ 编码表示法
 - ◆ 混合表示法
- 在微指令中还可附设一个常数字段
 - ◆ 可作为操作数送入ALU运算
 - ◆ 也可作为计数器初值用来控制微程序循环次数





直接表示法

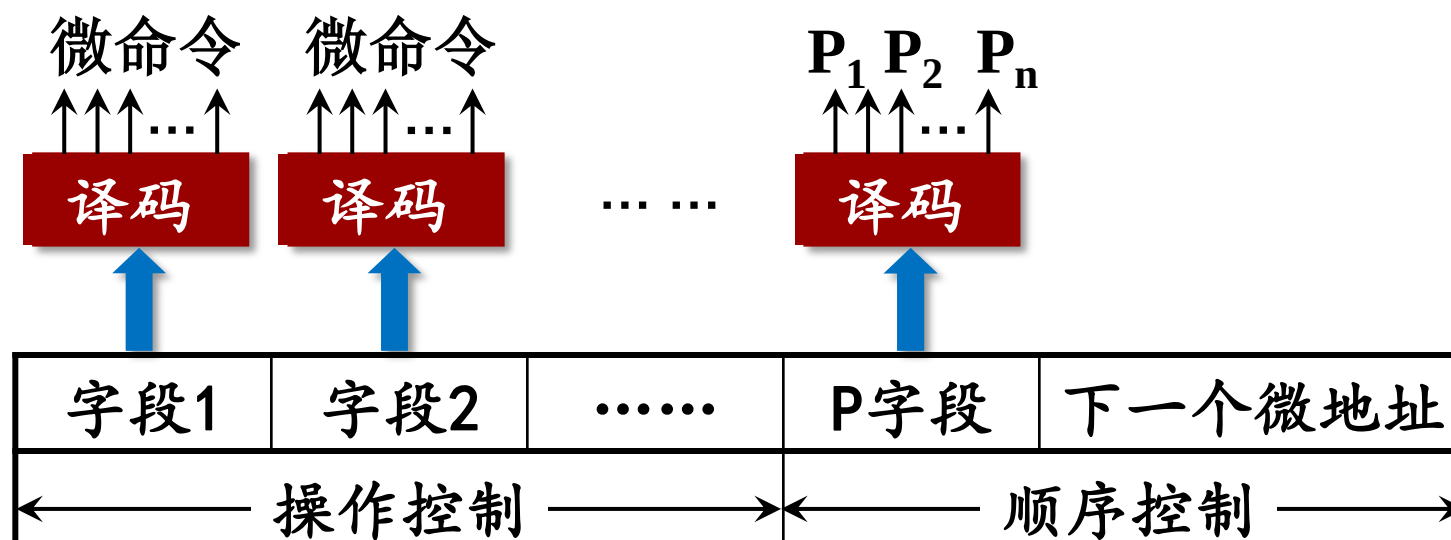
- 其特点是操作控制字段中的每一位代表一个微命令
- 优点：简单直观，其输出直接用于控制
- 缺点：微指令字较长，因而使控制存储器容量较大





编码表示法 (1)

- 把一组**相斥性**的微命令信号组成一个小组（即一个字段），然后通过小组（字段）译码器对每一个微命令信号进行译码，译码输出作为操作控制信号





编码表示法 (2)

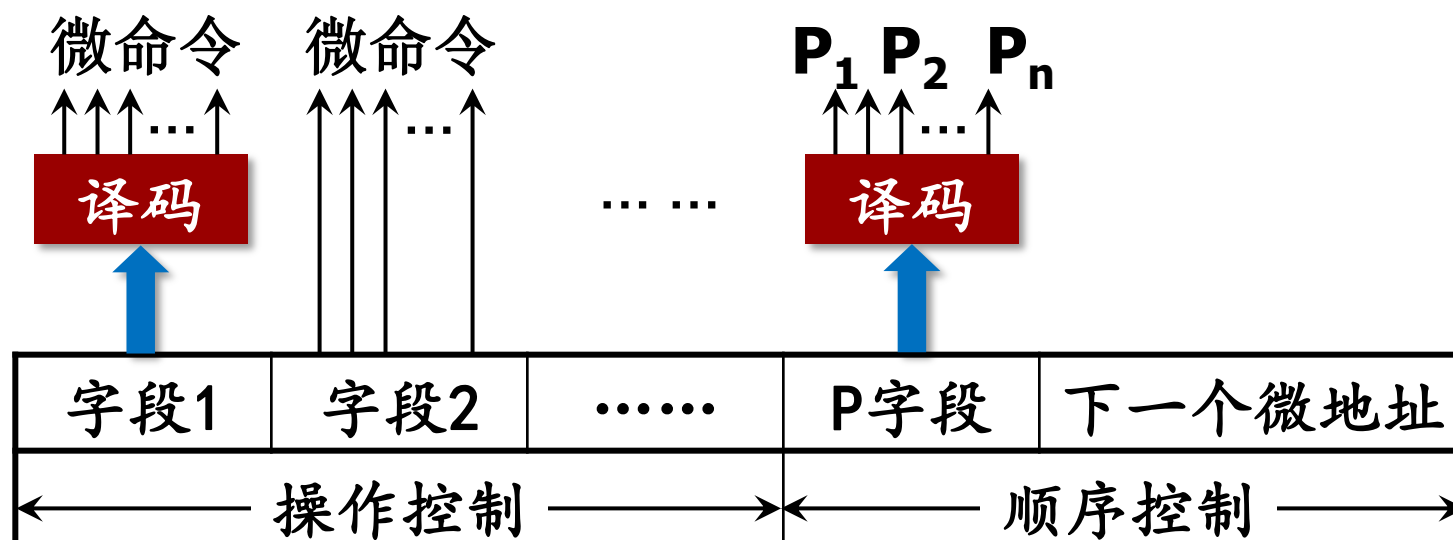
- 用较少的二进制位数表示较多的微命令信号， n 位二进制译码后可表示 2^n-1 个微命令
- 与直接控制法相比，编码表示法使微指令字的长度大大缩短。但由于增加译码电路，微程序的执行速度稍稍减慢
- 编码表示法使用较普遍





混合表示法

- 把直接表示法与字段编码法混合使用，以便能综合考虑指令字长、灵活性、执行微程序速度等方面的要求





2. 微地址形成方法

- 微指令执行顺序是靠执行当前微指令时生成下一条微指令地址来保证的
- 产生后继微地址的方法
 - ◆ 计数器方式
 - ◆ 多路转移方式
 - ◆ 由指令操作码转换得到。典型方式为查表方式，即在ROM中存放微地址映射表，用指令的操作码作为偏移量进行查表，查到的表项为该指令对应的微程序入口地址





计数器方式

■ 与程序计数器PC类似

- ◆ 在顺序执行微指令时，后继微地址由当前微地址加上一个增量来产生
- ◆ 在非顺序执行微指令时，必须通过转移方式，转去执行指定微地址的微指令

■ 微地址寄存器通常可视为计数器，且顺序执行的微指令序列必须安排在控制存储器的连续单元中

■ 特点

- ◆ 微指令的顺序控制字段较短，微地址产生机构简单
- ◆ 但是多路并行转移功能较弱，速度较慢，灵活性较差





多路转移方式

- 一条微指令具有多个转移分支的能力称为多路转移
- 当微程序不产生分支时，后继微地址直接由微指令的顺序控制字段给出
- 当微程序出现分支时，有若干“候选”微地址可供选择：根据顺序控制字段中P“判别测试”位和“状态条件”位来选择其中一个微地址
- 若“状态条件”有 n 位，可实现微程序 2^n 路转移，且涉及微地址寄存器的 n 位
- 特点
 - ◆ 能以较短的顺序控制字段配合，实现多路并行转移，灵活性好，速度较快
 - ◆ 转移地址逻辑需要用组合逻辑方法设计



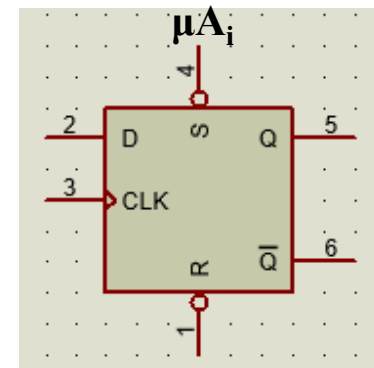


例2

微地址寄存器有6位 ($\mu A_5 - \mu A_0$)，当需要修改其内容时，可通过某一位触发器的SET端将其置“1”。现有三种情况：

- (1) 执行“取指”微指令后，微程序按IR的OP字段 ($IR_3 - IR_0$) 进行16路分支；
- (2) 执行条件转移指令微程序时，按进位标志C的状态进行2路分支；
- (3) 执行某控制台指令微程序时，按 IR_4 、 IR_5 的状态进行4路分支。

请按多路转移方法设计微地址转移逻辑。





例3解 (1)

解：按所给设计条件，微程序有三种判别测试方式，分别为 P_1 ， P_2 ， P_3 。由于修改 μA_5 - μA_0 内容具有很大灵活性，现分配如下：

(1)用 P_1 和 IR_3 - IR_0 修改 μA_3 - μA_0

(2)用 P_2 和 C 修改 μA_0

(3)用 P_3 和 IR_5 ， IR_4 修改 μA_5 ， μA_4





例3解 (2)

另外还要考虑时间因素 T_4 （假设CPU周期最后一个节拍脉冲），故转移逻辑表达式如下：

$$\mu A_5 = P_3 \cdot IR_5 \cdot T_4 \quad \mu A_4 = P_3 \cdot IR_4 \cdot T_4$$

$$\mu A_3 = P_1 \cdot IR_3 \cdot T_4 \quad \mu A_2 = P_1 \cdot IR_2 \cdot T_4$$

$$\mu A_1 = P_1 \cdot IR_1 \cdot T_4 \quad \mu A_0 = P_1 \cdot IR_0 \cdot T_4 + P_2 \cdot C \cdot T_4$$

由于是通过触发器SET端进行置“1”（通常低电平有效），所以前5个表达式需用“与非”门实现，最后一个用“与门”+“或非门”实现





3. 微指令格式

- 微指令的格式大体分成两类
 - ◆ 水平型微指令
 - ◆ 垂直型微指令





水平型微指令

- 一次能定义并执行多个并行操作微命令的微指令

控制字段	判别测试字段	下地址字段
------	--------	-------

- 按控制字段的编码方法不同可分为:
 - ◆ 全水平型（不译码法）微指令
 - ◆ 字段编码法水平型微指令
 - ◆ 直接表示和编码表示相混合的水平型微指令





垂直型微指令

- 微指令中设置**微操作码**字段，由微操作码规定微指令的功能
- 结构类似于机器指令的结构，它有操作码，在一条微指令中只有1~2个微操作命令，每条微指令的功能简单
- 实现一条机器指令的微程序要比水平型微指令编写的微程序长得多
- 特点：微程序较长，而每一条微指令较短





垂直型微指令举例（1）

微指令字长16位，微操作码3位

■ 寄存器 - 寄存器传送型微指令

15	13	12	8	7	3	2	0
000	源寄存器编址			目标寄存器编址			其他

- 功能：把源寄存器数据送目标寄存器
- 13-15位为微操作码，源寄存器和目标寄存器编址各为5位，可指定31个寄存器





垂直型微指令举例（2）

■ 运算控制型微指令

15	13	12	8	7	3	2	0
001		左输入源编址			右输入源编址		ALU

- 功能：选择ALU的左、右输入源信息，按ALU字段所指定的运算功能进行处理，并将结果送入暂存器中
- 有8种运算功能，左、右输入源编址可指定31种信息源之一





垂直型微指令举例（3）

■ 访问主存微指令

15	13	12	8	7	3	2	1	0
010	寄存器编址			存储器编址			读写	其他

- 功能：将主存中的一个单元的信息送入寄存器或者将寄存器的数据送往主存
- 存储器编址是按规定的寻址方式进行编址
- 第1、2位指定读操作或写操作（其中之一）





垂直型微指令举例（4）

■ 条件转移微指令

15	13	12	4	3	0
011			D		测试条件

- 功能：根据测试对象的状态决定是转移到D所指定的微地址单元，还是顺序执行下一条微指令
- 9位D字段不足以表示一个完整的微地址，但可以用来替代当前 μ PC的低位地址
- 测试条件字段有4位，可规定16种测试条件





水平型与垂直型微指令比较

- 水平型微指令并行操作能力强，效率高，灵活性强，垂直型微指令则较差
- 水平型微指令执行一条指令的时间短，垂直型微指令执行时间长
- 用水平型微指令解释指令的微程序，有微指令字较长而微程序短的特点，垂直型微指令则相反
- 水平型微指令用户难以掌握，而垂直型微指令与指令比较相似，相对来说，比较容易掌握





动手做

下列关于RISC的叙述中，错误的是

- A. RISC普遍采用微程序控制器
- B. RISC大多数指令在一个时钟周期内完成
- C. RISC的内部通用寄存器数量相对CISC多
- D. RISC的指令数、寻址方式和指令格式种类相对CISC少





例

某计算机采用微程序控制器，共有32条指令，公共的取指令微程序包含2条微指令，各指令对应的微程序平均由4条微指令组成，采用断定法（下址字段法）确定下条微指令的地址，则微指令中下址字段的位数至少是多少位？

解：解释32条指令，微程序中总共的微指令条数至少有：

$$32 \times 4 + 2 = 130$$

故：微指令中下址字段的位数至少需要8位





例

某计算机的控制器采用微程序控制方式，微指令中的控制字段采用字段直接编码法，共有33个微命令，构成5个互斥类，分别包含7、3、12、5和6个微命令，则操作控制字段至少有多少位？

解：操作控制字段共15位

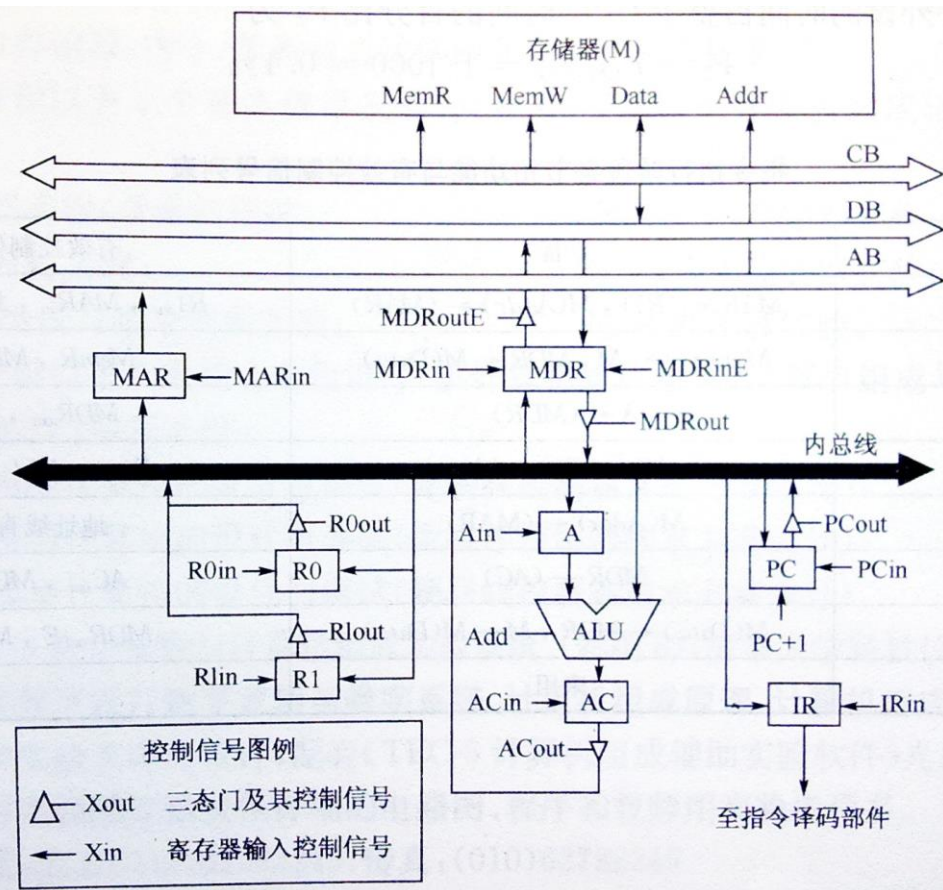
3	2	4	3	3
---	---	---	---	---





例题 (1)

某计算机字长16位，采用16位定长指令字结构，部分数据通路结构如右图所示，图中所有控制信号为1时表示有效、为0时表示无效，例如控制信号MDRinE为1表示允许数据从DB打入MDR，MDRin为1表示允许数据从内总线打入MDR。假设MAR的输出一直处于使能状态。加法指令“ADD (R1), R0”的功能为 $(R0) + ((R1)) \rightarrow (R1)$ ，即将R0中的数据与R1的内容所指主存单元的数据相加，并将结果送入R1的内容所指主存单元中保存。





例题 (2)

下表给出了上述指令取指和译码阶段每个节拍（时钟周期）的功能和有效控制信号，请按表中描述方式用表格列出指令执行阶段每个节拍的功能和有效控制信号。

时钟	功能	有效控制信号
C1	$MAR \leftarrow (PC)$	PCout, MARin
C2	$MDR \leftarrow M(MAR)$ $PC \leftarrow PC+1$	MemR, MDRinE PC+1
C3	$IR \leftarrow (MDR)$	MDRout, IRin
C4	指令译码	无





解答

时钟	功能	有效控制信号
C1	$MAR \leftarrow (R1), M(Addr) \leftarrow (MAR)$	R1out, MARin, 地址线直通
C2	$M(Data) \leftarrow M, MDR \leftarrow M(Data)$	MemR, MDRin
C3	$A \leftarrow (MDR)$	MDRout, Ain
C4	$AC \leftarrow (R0) + (A)$	R0out, Add, ACin
C1	$M(Addr) \leftarrow (MAR)$	地址线直通
C2	$MDR \leftarrow (AC)$	ACout, MDRin
C3	$M(Data) \leftarrow MDR, M \leftarrow M(Data)$	MDRoutE, MemW
C4	未用	

